SESAME
School of High-Tech, AI & Management

المدرسة العليا الخاصة للعلوم التطبيقية و إدارة الأعمال
ECOLE SUPÉRIEURE PRIVÉE DES SCIENCES APPLIQUÉES ET DE MANAGEMENT

RAPPORT DE STAGE DE FIN D'ÉTUDES

Intitulé du stage

Système de gestion des flux SEPA (ISO 20022) avec détection intelligente d'anomalies pour la paie HR Access

---

Réalisé par

Zina ElMorri

Société d'accueil

sopra hr SOFTWARE

Sopra HR Software

Encadrant entreprise

M. Mohamed Zied FIOUEL

Encadrant universitaire

M. Malek BOUZAIENE

Année universitaire

2025 / 2026

Dédicace

Je dédie ce travail à mes parents, qui m'ont soutenue à chaque étape de ce parcours avec une patience infinie, des sacrifices et une confiance inébranlable en moi, même dans les moments où je doutais de moi-même. Leur encouragement a été la force silencieuse derrière chaque nuit tardive et chaque défi que j'ai surmonté, et ils ont contribué à façonner la personne que je suis devenue.

À mes frères et sœurs, pour leurs encouragements constants, pour avoir célébré mes petites victoires comme si c'étaient les leurs, et pour avoir toujours été là dès que j'en avais besoin.

À mes grands-parents, dont l'amour et les prières m'ont portée plus loin qu'ils ne le sauront jamais.

À mes amis, qui se sont tenus à mes côtés dans les moments les plus difficiles, qui ont su m'écouter quand j'avais besoin de parler, et qui m'ont poussée vers l'avant.

Et à tous ceux qui ont fait partie de mon parcours de près ou de loin, y compris les enseignants, mentors et amis : votre soutien, votre gentillesse et vos encouragements signifient plus que les mots ne peuvent l'exprimer.

Zina Elmorri

Remerciements

Je tiens à exprimer ma profonde gratitude à Sopra HR Software pour m'avoir accueillie durant ce projet de fin d'études et pour la richesse de l'environnement humain et technique qui a entouré son développement. J'ai eu la chance de travailler aux côtés de professionnels expérimentés du domaine bancaire et des systèmes d'information RH, dont la disponibilité, la rigueur et la bienveillance ont profondément enrichi cette expérience.

Mes remerciements sincères vont à mon encadrant en entreprise, dont les conseils avisés sur la solution HR Access, sur les normes SEPA / ISO 20022 et sur les bonnes pratiques de développement Java Enterprise ont été déterminants pour orienter la conception et l'implémentation de la plateforme.

J'adresse également mes remerciements chaleureux à mon encadrant académique pour son suivi patient, ses relectures attentives et ses recommandations méthodologiques qui ont grandement contribué à la qualité de ce mémoire.

Je remercie l'ensemble des membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

Enfin, ma gratitude s'adresse à toutes les personnes qui, de près ou de loin, ont pris le temps de partager un conseil, une piste de résolution ou une remarque bienveillante. Chacune de ces contributions a joué un rôle dans l'aboutissement de ce projet.

Résumé

Le présent travail s'inscrit dans le cadre d'un projet de fin d'études réalisé au sein de la société Sopra HR Software en vue de l'obtention du diplôme national d'ingénieur à l'École Supérieure Privée des Sciences Appliquées et de Management (SESAME).

L'objectif de ce projet consiste à concevoir et implémenter, pour la solution de paie *HR Access*, une plateforme de gestion des virements de salaires au format SEPA (ISO 20022, messages `pain.001`). Contrairement aux approches classiques limitées à une version figée du schéma, la plateforme permet d'importer et de gérer dynamiquement plusieurs versions du schéma XSD (`pain.001.001.03` et `pain.001.001.09`), de générer le fichier XML correspondant à partir des paiements sélectionnés, et de le valider systématiquement avant son enregistrement.

La plateforme intègre également une détection hybride des anomalies métier, combinant un moteur de règles expertes (IBAN, BIC, montants, doublons) et un service d'intelligence artificielle dédié à la détection des montants statistiquement suspects, chaque anomalie étant accompagnée d'une recommandation actionnable. Un tableau de bord restitue les indicateurs clés d'activité, tandis qu'un mécanisme d'authentification et de gestion des rôles (administrateur, gestionnaire RH, validateur, auditeur) sécurise l'accès aux fonctionnalités, chaque action sensible étant tracée dans un journal d'audit.

L'application se compose d'un back-end Spring Boot exposant une API REST sécurisée par jeton JWT, d'une interface React et d'un module d'intelligence artificielle Python, l'ensemble étant conteneurisé avec Docker pour faciliter son déploiement.

Mots clés — HR Access, SEPA, ISO 20022, pain.001, virements de salaires, validation XSD, détection d'anomalies, intelligence artificielle, Spring Boot, React, PostgreSQL.

Table des matières

Introduction Générale **1**

1 Présentation du contexte du projet **3**
1.1 Introduction . . . 3
1.2 Présentation de la société d'accueil . . . 3
1.2.1 La solution HR Access . . . 4
1.3 Présentation du projet . . . 4
1.3.1 Problématique . . . 4
1.3.2 Analyse de l'existant . . . 5
1.3.3 Solution proposée . . . 6
1.3.4 Analyse des besoins . . . 7
1.4 Choix méthodologique . . . 9
1.4.1 Agile Scrum . . . 10
1.4.2 Application de la méthodologie . . . 11
1.5 Conclusion . . . 11

2 Sprint 0 : Étude préalable et choix des technologies **12**
2.1 Introduction . . . 12
2.2 Domaine métier : paie et virements SEPA . . . 12
2.2.1 L'espace SEPA et la norme ISO 20022 . . . 12
2.2.2 Structure d'un fichier pain.001 . . . 13
2.2.3 Identifiants bancaires : IBAN et BIC . . . 13
2.3 Détection intelligente d'anomalies : approche hybride . . . 14
2.4 Choix technologiques . . . 15
2.4.1 Langage et framework back-end . . . 15
2.4.2 Framework front-end . . . 16
2.4.3 Module d'intelligence artificielle . . . 17
2.4.4 Base de données . . . 17
2.4.5 Outils annexes et DevOps . . . 18
2.5 Backlog produit global . . . 18
2.6 Diagramme de cas d'utilisation global . . . 22

2.7 Diagramme de classes global 24
2.8 Diagramme de séquence global 26
2.9 Architecture logicielle cible 28
2.10 Conclusion 28

3 Sprint 1 : Authentification JWT, gestion des utilisateurs et journal d'audit 30
3.1 Introduction 30
3.2 Backlog du sprint 30
3.3 Diagrammes de conception 31
3.3.1 Diagramme de cas d'utilisation 31
3.3.2 Diagramme de classes 31
3.3.3 Diagramme de séquence – Authentification 31
3.3.4 Diagramme d'activités 32
3.4 Modèle de données 32
3.5 Implémentation 33
3.5.1 Configuration Spring Security 33
3.5.2 Service JWT 33
3.5.3 Filtre d'authentification 34
3.5.4 Journal d'audit 34
3.5.5 API REST du sprint 34
3.5.6 Mesures de sécurité mises en œuvre 34
3.5.7 Interface React 35
3.6 Conclusion 36

4 Sprint 2 : Gestion dynamique des XSD et génération des virements de masse SEPA 37
4.1 Introduction 37
4.2 Backlog du sprint 37
4.3 Diagrammes de conception 38
4.3.1 Diagramme de cas d'utilisation 38
4.3.2 Diagramme de classes 38
4.3.3 Diagramme de séquence 39
4.4 Scénario illustratif de bout en bout 39
4.5 Gestion dynamique des schémas XSD 40
4.5.1 Schéma XSD importé 40
4.5.2 Analyse dynamique d'un XSD 40
4.6 Génération dynamique du XML pain.001 41
4.6.1 Variantes supportées 41
4.6.2 Construction du fichier 41
4.6.3 Nettoyage SEPA 41
4.7 Validation XSD avec collecte exhaustive des erreurs 41

4.8 Historique immuable des fichiers générés 42
4.9 API REST du sprint 42
4.10 Interface React 42
4.11 Qualité : jeu de tests 43
4.12 Conclusion 43

5 Sprint 3 : Détection hybride des anomalies (règles métier + IA Python) et recommandations 44
5.1 Introduction 44
5.2 Backlog du sprint 45
5.3 Architecture de détection hybride 45
5.4 Moteur de règles métier Java 46
5.4.1 Types d'anomalies couverts 46
5.4.2 Validation IBAN MOD-97 46
5.4.3 Validation BIC ISO 9362 46
5.4.4 Détection des doublons 47
5.5 Service IA Python : détection des montants suspects 47
5.5.1 Features par paiement 47
5.5.2 Modèle Isolation Forest 47
5.5.3 Scoring et seuil de décision 48
5.5.4 Explication lisible 48
5.5.5 Endpoints du service IA 48
5.6 Intégration back-end $\rightarrow$ IA 48
5.6.1 Appel du service IA 48
5.6.2 Garanties de non-régression 48
5.6.3 Enrichissement de la transaction 49
5.7 Entité `Anomalie` et cycle de vie 49
5.8 Moteur de recommandations 49
5.9 Configuration et supervision 50
5.10 Interface React 50
5.11 Évaluation 51
5.11.1 Protocole d'évaluation 51
5.11.2 Métriques retenues 51
5.11.3 Résultats 52
5.11.4 Tests automatisés 52
5.12 Conclusion 52

6 Sprint 4 : Tableau de bord KPI, cycle de vie et conteneurisation 53
6.1 Introduction 53
6.2 Backlog du sprint 54

6.3 Tableau de bord KPI 54
6.3.1 Définitions 54
6.3.2 Indicateurs restitués 54
6.3.3 Calcul des indicateurs 55
6.3.4 Taux d'erreur 55
6.3.5 Endpoint REST 55
6.3.6 Interface React 56
6.4 Finalisation du cycle de vie des anomalies 56
6.5 Documentation OpenAPI / Swagger 56
6.6 Conteneurisation Docker 57
6.6.1 Dockerfiles multi-stages 57
6.6.2 Orchestration `docker-compose` 57
6.6.3 Variables d'environnement 57
6.6.4 Exploitation 57
6.7 Points d'accès finaux 58
6.8 Synthèse qualité 58
6.9 Bilan quantitatif 58
6.10 Limites du projet 59
6.11 Conclusion 60

Conclusion Générale **61**
Problème initial 61
Solution proposée 61
Réalisation technique 61
Résultats obtenus 62
Limites 63
Perspectives 63

Références **65**

Table des figures

1.1 Architecture générale de la plateforme SEPA intelligente 6
1.2 Méthodologie Scrum 10

2.1 Spring Boot 3 et Java 17 pour le back-end 16
2.2 React 18 et Vite pour le front-end 17
2.3 Diagramme de cas d'utilisation global 23
2.4 Diagramme de classes global 25
2.5 Diagramme de séquence global 27
2.6 Architecture logicielle en trois services conteneurisés 28

3.1 Cas d'utilisation du sprint 1 31
3.2 Diagramme de classes du sprint 1 31
3.3 Séquence d'authentification par JWT 32
3.4 Activités du processus d'authentification 32
3.5 Écran de connexion 35
3.6 Gestion des utilisateurs (vue administrateur) 35
3.7 Journal d'audit : traçabilité des actions 35

4.1 Cas d'utilisation du module XSD + virements de masse 38
4.2 Diagramme de classes du module SEPA 39
4.3 Séquence de génération d'un fichier SEPA 39
4.4 Analyse d'un XSD : arbre des éléments et contraintes 43
4.5 Virements de masse : filtres, aperçu coloré et conformité XSD 43

5.1 Architecture hybride de détection : règles Java + modèle Python 45
5.2 Interface Anomalies : règles + IA + recommandations 51
5.3 Détail d'une anomalie IA avec explications et recommandation 51

6.1 Tableau de bord KPI de la plateforme 56
6.2 Page d'accueil opérationnelle (statistiques et dernières anomalies) 56

Liste des tableaux

1.1 Évaluation des outils existants pour la génération et le contrôle des flux SEPA . 5
1.2 Rôles applicatifs et responsabilités 9
1.3 Comparaison des méthodologies traditionnelle (Cascade) et Agile Scrum 9
1.4 Backlog du projet et planification des sprints 11

2.1 Principaux blocs d'un fichier pain.001 pour les virements de salaires 13
2.2 Comparaison des deux approches de détection retenues conjointement 14
2.3 Comparaison des frameworks back-end 15
2.4 Comparaison des frameworks front-end 16
2.5 Comparaison des stacks IA 17
2.6 Comparaison des SGBD 18
2.7 Backlog produit global 19

3.1 Backlog du Sprint 1 30
3.2 Entités persistées du sprint 1 33
3.3 Endpoints REST du Sprint 1 34

4.1 Backlog du Sprint 2 38
4.2 Scénario illustratif de bout en bout de la gestion dynamique des XSD 40
4.3 Endpoints REST du Sprint 2 42

5.1 Backlog du Sprint 3 45
5.2 Types d'anomalies supportés et sources 46
5.3 Features par paiement attendus par le modèle 47
5.4 Niveaux de risque dérivés du score 48
5.5 Cycle de vie des anomalies 49
5.6 Extrait du catalogue de recommandations 50
5.7 Comparaison Isolation Forest vs seuil fixe sur le jeu de données synthétique . . 52

6.1 Backlog du Sprint 4 54
6.2 Indicateurs restitués par le tableau de bord 55
6.3 Endpoint du tableau de bord KPI 55

6.4 Services déclarés dans `docker-compose.yml` 57
6.5 URL exposées par la plateforme 58
6.6 Suites de tests livrées à la fin du sprint 4 58
6.7 Bilan quantitatif de la réalisation 59

Liste des abréviations

API	Interface de Programmation d'Applications (Application Programming Interface)
BCrypt	Fonction de hachage des mots de passe (Blowfish-based Crypt)
BIC	Code d'Identification Bancaire (Bank Identifier Code)
CI/CD	Intégration et Déploiement Continus (Continuous Integration / Continuous Deployment)
CORS	Partage de Ressources entre Origines Multiples (Cross-Origin Resource Sharing)
CRUD	Créer, Lire, Mettre à jour, Supprimer (Create, Read, Update, Delete)
DTO	Objet de Transfert de Données (Data Transfer Object)
EPC	European Payments Council
HTTPS	Protocole de Transfert Hypertexte Sécurisé (Hypertext Transfer Protocol Secure)
IA	Intelligence Artificielle (Artificial Intelligence)
IBAN	Numéro de Compte Bancaire International (International Bank Account Number)
ISO	Organisation Internationale de Normalisation (International Organization for Standardization)
JPA	Java Persistence API
JDBC	Java Database Connectivity
JVM	Machine Virtuelle Java (Java Virtual Machine)
JSON	Notation d'Objets JavaScript (JavaScript Object Notation)
JWT	Jeton Web JSON (JSON Web Token)
KPI	Indicateur Clé de Performance (Key Performance Indicator)
LOF	Facteur d'Isolement Local (Local Outlier Factor)
ML	Apprentissage Automatique (Machine Learning)
OIDC	OpenID Connect
REST	Transfert d'État de Représentation (Representational State Transfer)

RH	Ressources Humaines
SEPA	Espace Unique de Paiements en Euros (Single Euro Payments Area)
SGBD	Système de Gestion de Base de Données
SIRH	Système d'Information Ressources Humaines
SPA	Application Monopage (Single Page Application)
SQL	Langage de Requêtes Structuré (Structured Query Language)
TLS	Sécurité de la Couche de Transport (Transport Layer Security)
UML	Langage de Modélisation Unifié (Unified Modeling Language)
URL	Localisateur Uniforme de Ressource (Uniform Resource Locator)
XML	Langage de Balisage Extensible (Extensible Markup Language)
XSD	Définition de Schéma XML (XML Schema Definition)
XXE	Attaque par Entité Externe XML (XML External Entity)

Introduction Générale

Dans un environnement où la digitalisation des processus de gestion des ressources humaines s'est généralisée, le versement des salaires est devenu un processus totalement dématérialisé, échangé entre le système d'information RH et les partenaires bancaires sous la forme de fichiers de paiement structurés. La norme européenne SEPA (Single Euro Payments Area), déclinée à travers le standard ISO 20022, s'est imposée comme la référence pour ces échanges : les ordres de virements de salaires prennent la forme de messages XML `pain.001` (*Customer Credit Transfer Initiation*) dont la structure est contrainte par un schéma XSD publié par l'EPC (European Payments Council). Plusieurs variantes coexistent en production (`pain.001.001.03` encore largement répandue, `pain.001.001.09` plus récente), chacune avec son propre jeu d'éléments, d'attributs et de contraintes de type.

Les solutions de paie de référence, telles que *HR Access*, produisent chaque mois plusieurs milliers de lignes de paiement destinées à être transformées en fichier SEPA puis transmises aux systèmes bancaires pour exécution. Ce processus critique est particulièrement sensible aux erreurs : un IBAN mal saisi, un BIC incorrect, un montant aberrant ou un doublon de bénéficiaire peuvent entraîner le rejet du fichier par la banque, un versement erroné, voire un litige avec le salarié. Les contrôles historiquement mis en place restent essentiellement techniques (vérification du format XML par rapport au schéma XSD) et n'intègrent pas la dimension métier des données de paie, qui reste examinée manuellement par les gestionnaires RH. L'étude des solutions existantes menée dans le cadre de ce projet (voir chapitre 1) n'a par ailleurs pas identifié d'outil combinant simultanément une *gestion dynamique* des schémas XSD, une *génération* contrainte par ces schémas, une *détection intelligente* des anomalies métier et une *traçabilité* complète des actions réalisées sur chaque ordre de paiement.

Ce constat justifie la mise en place d'une plateforme qui automatise l'ensemble du cycle : import et analyse des schémas XSD officiels, génération du fichier SEPA ajustée à la version du schéma choisi, validation technique systématique, détection hybride des anomalies métier (règles expertes et modèle d'apprentissage automatique) et pilotage par indicateurs clés de performance. C'est dans ce cadre que s'inscrit ce projet de fin d'études, réalisé au sein de la société **Sopra HR Software** en vue de l'obtention du diplôme national d'ingénieur à l'École Supérieure Privée des Sciences Appliquées et de Management (SESAME).

L'objectif de ce travail est de concevoir et de développer une plateforme qui doit : (i) gérer dynamiquement les schémas XSD (import, versions, analyse des contraintes), (ii) produire un

fichier XML `pain.001` conforme à la variante choisie, (iii) valider ce fichier au niveau structurel contre le XSD officiel en collectant toutes les erreurs, (iv) détecter les anomalies métier au moyen d'un moteur de règles expertes (validation des IBAN et des BIC, montants, doublons, net supérieur au brut) complété par un service d'intelligence artificielle dédié aux montants statistiquement suspects, (v) proposer, pour chaque anomalie, une *recommandation lisible* issue d'un catalogue, (vi) tracer intégralement les actions au moyen d'un journal d'audit, (vii) piloter l'activité à travers un tableau de bord KPI (fiches générées, taux d'erreurs bloquantes, évolution sur douze mois, top clients, répartition par type/source/statut) et (viii) offrir une gestion fine des rôles utilisateurs (Administrateur, Gestionnaire RH, Validateur, Auditeur).

Le présent rapport est organisé en six chapitres. Le chapitre 1 présente le contexte général du projet, la société d'accueil, la problématique métier autour du traitement des paiements de salaires, l'analyse de l'existant, la solution proposée et la méthodologie Scrum retenue pour la conduite du projet. Le chapitre 2 correspond au sprint 0 : il expose les concepts fondamentaux du domaine et justifie les choix technologiques. Les chapitres 3 à 6 sont structurés selon les sprints de développement : authentification JWT, gestion des utilisateurs/rôles et journal d'audit (sprint 1) ; gestion dynamique des schémas XSD et génération des virements de masse SEPA (sprint 2) ; détection hybride des anomalies (règles métier Java + modèle IA Python) et moteur de recommandations (sprint 3) ; tableau de bord KPI, cycle de vie des anomalies, conteneurisation et déploiement (sprint 4). Enfin, une conclusion générale synthétise les résultats obtenus, les difficultés rencontrées et les perspectives d'évolution de la plateforme.

Chapitre 1

Présentation du contexte du projet

1.1 Introduction

Ce chapitre d'ouverture pose le cadre du projet en exposant les éléments fondamentaux qui l'ont façonné. Il présente d'abord la société d'accueil et ses activités, puis introduit la solution de paie *HR Access*, cœur métier du sujet, ainsi que la problématique liée au traitement des flux de paiement des salaires. Il expose ensuite une analyse comparative des solutions existantes, formalise la solution proposée et détaille les besoins fonctionnels et non fonctionnels, avant de justifier le choix de la méthodologie de conduite de projet retenue.

1.2 Présentation de la société d'accueil

Sopra HR Software est l'éditeur de la suite *HR Access*, une solution de gestion des ressources humaines et de paie déployée par de très nombreuses organisations en France et à l'international. Filiale du groupe Sopra Steria, leader européen de la transformation numérique présent dans une trentaine de pays et rassemblant plus de cinquante mille collaborateurs, Sopra HR Software bénéficie de cette expertise reconnue dans les secteurs bancaire, financier, industriel et administration publique.
Cette solution assure la tenue du dossier administratif du salarié, la gestion des temps, le calcul de la paie et la production des déclarations sociales et fiscales. L'entreprise a mis à ma disposition un environnement de développement complet et un accompagnement technique tout au long de la réalisation de ce projet.

Logo Sopra HR Software (éditeur de HR Access)

1.2.1 La solution HR Access

HR Access est une suite intégrée de gestion des ressources humaines qui couvre l'ensemble du cycle de vie du salarié dans l'entreprise. Elle intègre un moteur de paie paramétrable, un module de gestion administrative des dossiers salariés, un module de gestion des temps et des activités, ainsi qu'un module de production des flux à destination des organismes tiers (banques, URSSAF, mutuelles, etc.). Le versement effectif des salaires s'effectue à travers la génération d'un fichier de virements au format SEPA (ISO 20022), transmis à la banque du client pour exécution. C'est précisément sur cette dernière étape (la préparation), la validation et le contrôle qualité des fichiers SEPA que se concentre le projet.

1.3 Présentation du projet

Cette section décrit le projet, ses objectifs, son périmètre et les problèmes qu'il ambitionne de résoudre pour les utilisateurs de la solution HR Access.

1.3.1 Problématique

Chaque cycle de paie, les données consolidées sont transformées en un fichier XML SEPA `pain.001` qui décrit précisément l'ordre de virement adressé à la banque. Ce fichier doit respecter trois niveaux de conformité :

- **Conformité structurelle :** le XML doit se plier au schéma XSD officiel publié par l'European Payments Council. Plusieurs variantes sont admises par les banques (`pain.001.001.03`, `pain.001.001.09`, ...), chacune introduisant ses propres éléments, attributs et contraintes de longueur. Toute déviation entraîne un rejet immédiat par le canal bancaire.
- **Conformité réglementaire :** les codes IBAN et BIC doivent respecter les règles ISO 13616 (IBAN, contrôle MOD-97) et ISO 9362 (BIC, format alphanumérique 8 ou 11).
- **Conformité métier :** les montants doivent être strictement positifs et plausibles, les bénéficiaires correctement renseignés, le net inférieur au brut, et il ne doit pas exister de doublons de paiement pour un même salarié sur la même période.

Dans les processus actuels, la génération et le contrôle du fichier reposent en grande partie sur des scripts internes et sur une vérification humaine effectuée par les gestionnaires RH. Les contrôles automatiques mis en œuvre restent essentiellement techniques : ils s'assurent que le fichier est bien formé et respecte le XSD, mais ne remontent pas les anomalies métier qui n'apparaîtront qu'après réception du fichier par la banque, voire lorsque le salarié constate que son virement est erroné ou absent. Par ailleurs, les scripts existants ciblent une version figée du schéma, ce qui oblige à redévelopper une partie du code chaque fois qu'une banque demande une version plus récente. L'absence combinée de contrôles métier automatisés et de *souplesse sur la variante XSD* se traduit concrètement par des rejets bancaires, des ordres de re-génération,

des délais de paiement allongés et une charge de support importante pour les équipes RH. Enfin, aucune traçabilité fine (audit trail) ni aucun tableau de bord de pilotage n'est disponible pour mesurer la qualité du processus mensuel.
Le projet vise donc à répondre à la question suivante : *comment industrialiser la génération des fichiers SEPA au sein d'une plateforme web unifiée, en garantissant à la fois le respect dynamique du schéma XSD choisi par le client, la détection intelligente des anomalies métier avant envoi, la traçabilité des actions et le pilotage opérationnel du processus de paiement ?*

1.3.2 Analyse de l'existant

Avant de concevoir la plateforme, une analyse des solutions actuellement utilisées pour la génération et le contrôle des fichiers SEPA a été menée. Le Tableau 1.1 résume les outils les plus couramment rencontrés, leurs capacités et leurs limites au regard des besoins exprimés.

Critère	**Scripts internes HR Access**	**Modules SEPA bancaires (ex. Sage, Cegid)**	**Validateurs XSD génériques**
Automatisation de bout en bout	Partielle	Élevée mais couplée à l'ERP	Aucune
Génération pain.001 multi-versions	Non (une variante codée en dur)	Oui, mais fermée	Non
Validation XSD ISO 20022	Manuelle	Intégrée	Oui
Contrôles IBAN (MOD-97)	Non systématiques	Oui	Non
Contrôles BIC (ISO 9362)	Non	Oui	Non
Détection de doublons métier	Non	Limitée	Non
Contrôle sur montants aberrants (IA)	Non	Non	Non
Journal d'audit & traçabilité	Non	Partiel	Non
Tableau de bord KPI	Non	Partiel	Non
Interface RH dédiée	Non	Payante et propriétaire	Non
Ouverture API REST & conteneurisation	Faible	Faible (SI verrouillé)	N/A
Coût	Interne	Élevé (licence + maintenance)	Faible

TABLE 1.1 – Évaluation des outils existants pour la génération et le contrôle des flux SEPA

L'analyse fait ressortir plusieurs manques : les scripts internes de HR Access assurent la production du XML mais laissent les contrôles métier à la charge des gestionnaires et ne gèrent qu'une version figée du schéma ; les modules bancaires du marché offrent des contrôles riches mais sont propriétaires, coûteux et couplés à un ERP tiers ; enfin, les validateurs XSD publics ne contrôlent que la structure et ignorent totalement la dimension métier. L'étude comparative

réalisée dans le cadre de ce projet n'a pas identifié de solution répondant simultanément à l'ensemble des critères retenus : la *gestion dynamique* du schéma XSD, la génération contrainte par ce schéma, la validation structurelle, la détection intelligente des anomalies (règles expertes et apprentissage automatique), la traçabilité par audit trail et le pilotage par KPI dans une interface unifiée destinée aux gestionnaires RH utilisateurs de HR Access. Cette analyse reste circonscrite aux outils effectivement examinés et ne constitue pas une étude exhaustive du marché.

1.3.3 Solution proposée

Pour répondre à ces limitations, le projet propose une plateforme web modulaire composée de trois services conteneurisés qui coopèrent via des API REST. La Figure 1.1 illustre l'architecture générale de la solution.

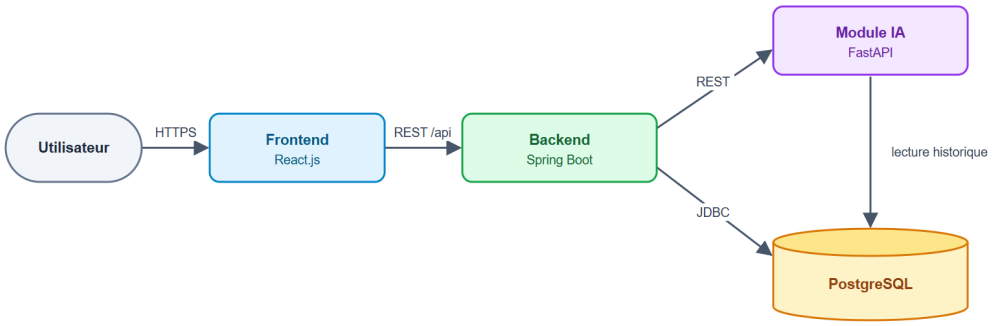


FIGURE 1.1 – Architecture générale de la plateforme SEPA intelligente

Notre solution repose sur une **architecture web en couches**, complétée par un **service d'intelligence artificielle indépendant**, comme l'illustre la figure 1.1. :
La **couche de présentation** est assurée par une application **React.js**. Elle constitue le point d'entrée de l'utilisateur, qui y accède via le protocole sécurisé **HTTPS**. Elle prend en charge l'affichage et l'interaction, puis transmet les requêtes au serveur au moyen d'une **API REST**.
La **couche applicative** est implémentée avec **Spring Boot**. Ce backend suit une **organisation en couches** :

- **Contrôleurs** : ils exposent les points d'accès REST et reçoivent les requêtes du frontend.
- **Services** : ils portent la logique métier.
- **Accès aux données** : les *repositories* Spring Data JPA gèrent les interactions avec la base de données, au-dessus de l'implémentation Hibernate.

Cette séparation des responsabilités facilite la lisibilité du code, les tests et la maintenance.
La **couche de données** repose sur un système de gestion de base de données relationnelle **PostgreSQL**. Le backend y accède via **Spring Data JPA / Hibernate**, qui s'appuie en interne sur un pilote **JDBC** pour dialoguer avec le SGBD ; la figure 1.1 représente cette communication au niveau protocole (JDBC), tandis que le code applicatif manipule des entités JPA.

En parallèle de ces couches, les traitements d'intelligence artificielle sont confiés à un **service séparé** développé avec **FastAPI** (Python). Le backend l'appelle via des requêtes **REST**, et il consulte directement la base de données en **lecture** afin d'exploiter les **données historiques** nécessaires à ses traitements. Ce découplage permet de faire évoluer, de déployer et de mettre à l'échelle le module IA indépendamment du reste de l'application. Il permet aussi de tirer parti de l'écosystème Python, particulièrement riche dans le domaine de l'apprentissage automatique. Les trois composants applicatifs (back-end, front-end, service IA) sont par ailleurs packagés et déployés sous forme de **services conteneurisés** (voir chapitre 6).

1.3.4 Analyse des besoins

1.3.4.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit accomplir :

- **Authentification et gestion des utilisateurs :** connexion par JWT, gestion des rôles (Administrateur, Gestionnaire RH, Validateur, Auditeur), encodage facial optionnel pour une authentification biométrique.
- **Journal d'audit :** enregistrement de toutes les actions sensibles (login, import XSD, génération, validation, changement de statut d'anomalie) avec auteur, IP, horodatage et résultat.
- **Gestion des schémas XSD :** import de nouveaux schémas, versions multiples, analyse dynamique des éléments (version, namespace, ordre, obligatoires, `maxLength`, `pattern`, énumérations).
- **Génération SEPA :** production d'un fichier XML `pain.001` conforme à la variante XSD choisie, incluant `GrpHdr`, `PmtInf` et n transactions `CdtTrfTxInf`, avec `NbOfTxs`/`CtrlSum` en `BigDecimal`.
- **Validation XSD :** contrôle structurel du fichier généré avec collecte exhaustive de toutes les erreurs (ligne, colonne, message traduit).
- **Détection d'anomalies métier :** contrôle des IBAN (MOD-97), des BIC (ISO 9362), des montants (zéro, négatif), des doublons, du cas net > brut, des champs obligatoires manquants.
- **Détection par IA :** identification des montants statistiquement suspects par Isolation Forest sur un vecteur de features par paiement.
- **Recommandations lisibles :** moteur de catalogue qui, pour chaque type d'anomalie, produit une recommandation actionnable incluant les vraies valeurs (matricule, IBAN masqué, montant).
- **Cycle de vie des anomalies :** statuts OUVERTE / CORRIGÉE / IGNORÉE avec traçabilité des changements.

— **Virements de masse :** sélection par société/période/matricules, aperçu coloré, génération avec validation, enregistrement en historique, retéléchargement à l'identique.

— **Tableau de bord KPI :** fiches générées, nombre d'erreurs bloquantes, taux d'erreurs, évolution sur douze mois, top 5 des clients à anomalies, répartitions par type/source/statut.

1.3.4.2 Besoins non fonctionnels

Les besoins non fonctionnels caractérisent la qualité attendue du système :

— **Fiabilité :** chaque fichier généré doit être conforme au schéma XSD choisi et exempt d'anomalies bloquantes avant tout téléchargement ; un re-téléchargement renvoie exactement le même contenu que l'enregistrement initial.

— **Sécurité :** authentification JWT, hachage BCrypt des mots de passe, contrôle CORS, échanges HTTPS, sécurité XXE (doctype interdit), journal d'audit, principe du moindre privilège applicatif via rôles.

— **Performance :** la génération et la validation d'un lot de plusieurs milliers de paiements doivent s'exécuter en quelques secondes ; les KPI sont calculés en lecture seule par agrégations SQL.

— **Portabilité :** l'application doit pouvoir fonctionner sur PostgreSQL en production et sur H2 en développement, grâce aux abstractions JPA/Hibernate ; l'ensemble est conteneurisé pour un démarrage reproductible.

— **Maintenabilité :** architecture en couches, séparation nette entre contrôleurs, services et repositories, DTO explicites pour l'API REST, interrupteur permettant de désactiver l'IA sans impacter le traitement.

— **Ergonomie :** interface React 18 unifiée, réactive et bilingue-ready, aperçu coloré du XML avec numéros de ligne, surlignage des erreurs XSD.

— **Observabilité :** documentation OpenAPI/Swagger générée automatiquement, logs structurés, interrupteurs côté IA.

1.3.4.3 Acteurs du système

Conformément à la gestion des rôles implémentée dans le module de sécurité, quatre acteurs humains et un acteur système interagissent avec la plateforme (Tableau 1.2).

Rôle	Responsabilités
ADMINISTRATEUR	Supervise l'application : création/désactivation des utilisateurs, import et retrait des schémas XSD, supervision du module IA (health/train), accès complet au journal d'audit et aux KPI.
GESTIONNAIRE_RH	Prépare les lots de paiements, choisit le XSD, lance la génération et la validation, consulte les anomalies, exporte le fichier SEPA.
VALIDATEUR	Vérifie les fiches en attente et approuve les paiements ; peut consulter en lecture les XSD, les virements de masse et les anomalies.
AUDITEUR	Lecture seule du journal d'audit et des indicateurs ; aucun droit d'écriture ni de génération.
Module IA	Service applicatif Python/FastAPI qui attribue un score de risque à chaque paiement et peut être ré-entraîné sur demande de l'administrateur.

TABLE 1.2 – Rôles applicatifs et responsabilités

Le système bancaire externe reste destinataire des fichiers SEPA XML pour exécution des virements, en dehors du périmètre applicatif de la plateforme.

1.4 Choix méthodologique

Le choix d'une méthodologie adaptée est essentiel pour organiser la construction incrémentale d'une plateforme qui combine plusieurs domaines (paie, virements, interface web, moteur de règles, IA, DevOps). Le Tableau 1.3 compare les deux approches candidates.

Critère	Traditionnel (Cascade [1])	Agile Scrum [2]
Approche	Linéaire, séquentielle	Itérative, incrémentale
Flexibilité	Rigide ; changements coûteux	Très flexible
Phases	Uniques et séquentielles	Sprints (1 à 4 semaines)
Implication du client	Début et fin	Continue
Exigences	Fixes au départ	Évolutives
Livraison	Produit final unique	Incréments fréquents
Gestion des risques	Traitée tôt	Continue
Documentation	Exhaustive dès le départ	Juste-à-temps
Visibilité	Faible jusqu'à la fin	Élevée à chaque sprint
Adaptation aux besoins évolutifs	Faible	Élevée

TABLE 1.3 – Comparaison des méthodologies traditionnelle (Cascade) et Agile Scrum

L'analyse fait clairement ressortir Agile Scrum comme la méthodologie la plus adaptée. La construction d'une plateforme SEPA touche des domaines techniques hétérogènes (standards ISO 20022, XSD, règles métier, modèle d'IA Python, interface React, conteneurisation), dont la complète formalisation en amont serait risquée. Une approche itérative permet au contraire de livrer des incréments fonctionnels démontrables, d'obtenir des retours rapides de l'enca-

drant métier et d'ajuster progressivement les règles de détection au regard des données réelles observées.

1.4.1 Agile Scrum

Agile Scrum [2, 3] est un cadre itératif et incrémental qui favorise la flexibilité, la collaboration et la satisfaction du client à travers des rôles clairement définis. La Figure 1.2 illustre le cycle de vie d'un sprint Scrum.

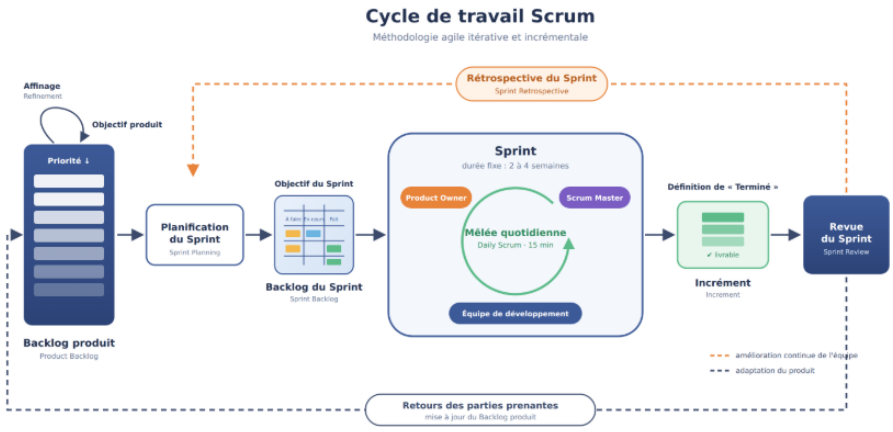


FIGURE 1.2 – Méthodologie Scrum

Les principaux éléments de Scrum adoptés dans ce projet sont :

- **Product Owner :** définit et priorise le backlog, représente la voix du client métier HR Access.
- **Scrum Master :** facilite les cérémonies Scrum et lève les obstacles.
- **Équipe de développement :** livre les incréments à chaque sprint.
- **Sprint Planning :** sélection des User Stories du backlog et engagement de l'équipe sur l'objectif du sprint.
- **Daily Scrum :** synchronisation quotidienne, partage des avancements et obstacles.
- **Sprint Review et Rétrospective :** démonstration du produit livré aux parties prenantes et amélioration continue du processus.

1.4.2 Application de la méthodologie

Le projet a été découpé en cinq sprints (un sprint 0 d'étude puis quatre sprints de réalisation). Le backlog global est présenté dans le Tableau 1.4.

Sprint	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Fonctionnalités	Authentification JWT, gestion des utilisateurs & rôles, journal d'audit	Gestion dynamique des schémas XSD et virements de masse SEPA pain.001	Détection d'anomalies hybride (règles métier Java + IA Python) et recommandations	Tableau de bord KPI, cycle de vie des anomalies, conteneurisation Docker
User Stories	En tant qu'utilisateur, je veux me connecter avec un rôle et retrouver mes actions dans le journal d'audit.	En tant que gestionnaire RH, je veux importer un XSD, voir ses contraintes, générer et valider un fichier SEPA conforme.	En tant que gestionnaire RH, je veux que les anomalies métier et les montants statistiquement anormaux me soient signalés avec des recommandations claires.	En tant qu'administrateur, je veux piloter l'activité à travers des KPI, suivre le cycle de vie des anomalies et déployer l'ensemble avec Docker.

TABLE 1.4 – Backlog du projet et planification des sprints

1.5 Conclusion

Ce chapitre a présenté la société Sopra HR Software, sa solution de paie HR Access et le contexte métier dans lequel s'inscrit le projet. Il a identifié la problématique du traitement des flux SEPA de paiement des salaires, les limites des solutions existantes et la solution proposée sous forme d'une plateforme web intégrée en trois services conteneurisés combinant génération dynamique, validation XSD multi-versions, détection hybride règles + IA, recommandations lisibles et pilotage par KPI. Le choix d'une conduite Agile Scrum a été justifié et le backlog des sprints détaillé. Les chapitres suivants approfondissent, sprint par sprint, la conception et la réalisation de la solution.

Chapitre 2

Sprint 0 : Étude préalable et choix des technologies

2.1 Introduction

Ce deuxième chapitre présente le sprint 0 : une étude préalable qui pose les concepts fondamentaux du domaine (paie, SEPA, ISO 20022, XSD, IBAN, BIC) et sélectionne les technologies utilisées lors de l'implémentation. L'objectif est d'identifier les briques les plus adaptées aux exigences fonctionnelles et non fonctionnelles d'une plateforme en trois services (back-end, front-end et module d'intelligence artificielle), orchestrée par conteneurs.

2.2 Domaine métier : paie et virements SEPA

Le versement des salaires constitue, dans toute entreprise, un processus critique dont la fiabilité conditionne la relation de confiance avec le salarié. À l'issue du cycle de paie, chaque ligne de paiement calculée par le moteur de paie est consolidée en une transaction qui comporte, notamment, l'identifiant de l'employé, l'identifiant du client (entreprise), le nom du bénéficiaire, l'IBAN, le BIC, le montant, la devise, le libellé et le statut (`BROUILLON`, `VALIDEE`, `APPROUVEE`, `REJETEE`, `SUPPRIMEE`). Seules les transactions au statut `APPROUVEE` sont consommées par la génération SEPA et alimentent le fichier `pain.001` transmis à la banque.

2.2.1 L'espace SEPA et la norme ISO 20022

L'espace SEPA (Single Euro Payments Area) est un espace de paiement unifié couvrant les pays de l'Union européenne ainsi que plusieurs pays associés. Il définit un cadre technique commun pour les virements et prélèvements en euros, avec pour objectif de traiter tous les paiements transfrontaliers avec la même simplicité qu'un paiement domestique.
Sur le plan technique, SEPA s'appuie sur le standard ISO 20022 [4], une norme internationale pour la description structurée des messages financiers. ISO 20022 définit un dictionnaire de

composants métier et un format XML, contraint par des schémas XSD [5], pour l'échange de messages entre les acteurs financiers. Le message utilisé pour un ordre de virement de salaires est `pain.001` : *Customer Credit Transfer Initiation*. Plusieurs versions de ce message coexistent en production, dont les deux qui doivent être supportées par le projet : `pain.001.001.03` (encore largement acceptée par les banques domestiques) et `pain.001.001.09` (version plus récente alignée sur les dernières spécifications EPC [6]).

2.2.2 Structure d'un fichier pain.001

Un message `pain.001` est structuré autour de trois blocs principaux :

Élément	Description
GrpHdr *Group Header*	En-tête du fichier de virements de salaires. Il regroupe l'identifiant du message (MsgId), la date et l'heure de création (CreDtTm), le nombre de transactions (NbOfTxs), le montant total des virements (CtrlSum) et le donneur d'ordre (InitgPty).
PmtInf *Payment Information*	Informations communes au lot de paiements : service SEPA (PmtTpInf/SvcLvl/Cd), catégorie SALA correspondant aux salaires, date d'exécution (ReqdExctnDt), entreprise débitrice (Dbtr), IBAN de son compte (DbtrAcct/Id/IBAN) et banque du débiteur (DbtrAgt).
CdtTrfTxInf *Credit Transfer Transaction Information*	Détails de chaque virement individuel à un salarié : identifiant de transaction (EndToEndId), montant et devise (InstdAmt), salarié bénéficiaire (Cdtr), IBAN de son compte (CdtrAcct/Id/IBAN), banque du bénéficiaire (CdtrAgt/FinInstnId/BIC ou BICFI selon la version) et libellé du virement (RmtInf/Ustrd).

TABLE 2.1 – Principaux blocs d'un fichier pain.001 pour les virements de salaires

À titre d'exemple, la version `pain.001.001.09` utilise la balise `<BICFI>` pour identifier la banque. La date d'exécution est renseignée dans l'élément `<Dt>`, à l'intérieur du bloc `<ReqdExctnDt>`, par exemple `<Dt>2026-08-01</Dt>`. La plateforme devra donc générer son XML *dynamiquement* en fonction du schéma XSD retenu par l'utilisateur. Chaque bloc est contraint par un schéma XSD publié par l'EPC. Toute déviation (élément manquant, ordre incorrect, type de donnée non conforme) provoque le rejet du fichier par la banque, d'où la nécessité d'une validation systématique en amont.

2.2.3 Identifiants bancaires : IBAN et BIC

Deux identifiants sont au cœur de tout virement SEPA :

— **IBAN** (International Bank Account Number, ISO 13616 [7]) : de 15 à 34 caractères alphanumériques commençant par un code pays sur deux lettres et deux chiffres de contrôle. La validité d'un IBAN se vérifie via l'algorithme *MOD 97* présenté ci-dessous.

— **BIC** (Business Identifier Code, ISO 9362 [8]) : 8 ou 11 caractères alphanumériques identifiant l'établissement bancaire, structurés en code banque (4 lettres), code pays (2 lettres), code emplacement (2 caractères) et éventuellement code d'agence (3 caractères).

2.2.3.1 Algorithme MOD 97 de validation de l'IBAN

La validation de l'IBAN repose sur le calcul suivant :

1. Déplacer les quatre premiers caractères (code pays + clé) à la fin de la chaîne.
2. Convertir chaque lettre en un entier (A=10, B=11, …, Z=35) et concaténer les chiffres obtenus.
3. Calculer le reste modulo 97 de ce grand entier.
4. L'IBAN est valide si et seulement si le reste vaut 1 :

$$\text{IBAN valide} \iff N \bmod 97 = 1$$

Cet algorithme est utilisé à la fois par le moteur de détection d'anomalies et par le nettoyage SEPA.

2.2.3.2 Format du BIC

Le format du BIC est validé contre la structure normalisée ISO 9362 (8 ou 11 caractères, code banque, pays et localisation, suffixe d'agence optionnel).

2.3 Détection intelligente d'anomalies : approche hybride

Le projet combine deux approches complémentaires de détection, chacune ayant son terrain de prédilection (Tableau 2.2).

Critère	**IA symbolique (règles métier en Java)**	**Apprentissage automatique (scikit-learn)**
Explicabilité	Totale (chaque anomalie porte le nom de la règle violée)	Modérée (score + raisons textuelles)
Besoin de données étiquetées	Aucun	Faible (apprentissage non supervisé)
Conformité réglementaire	Adaptée : règles alignées ISO 13616 et ISO 9362	Nécessite une justification textuelle
Détection de nouveaux motifs	Non	Oui (Isolation Forest, LOF, z-score)
Précision sur les règles connues	100 %	Variable
Mise en production	Immédiate dans le back-end	Service Python dédié, versionnable

TABLE 2.2 – Comparaison des deux approches de détection retenues conjointement

Compte tenu du caractère très structuré des règles à appliquer (IBAN, BIC, montants, doublons, net > brut) et de l'exigence d'explicabilité vis-à-vis du gestionnaire RH, le **moteur de règles symboliques Java** constitue le socle de la détection. Pour couvrir la famille des anomalies plus faiblement caractérisées (par exemple un montant qui s'écarte fortement de la médiane historique du salarié ou du client), un **module IA Python** dédié est mis en place. Son modèle de production est un *Isolation Forest* entraîné sur un vecteur de six features comportementales par paiement (écart à la moyenne du salarié sur six mois, z-score par client, variation vs mois précédent, changement récent d'IBAN, nombre de paiements sur un même IBAN, montant). Un second pipeline expérimental, combinant Isolation Forest, Local Outlier Factor et un score statistique *z-score* par une moyenne pondérée, sert de validation croisée pour comparer les approches, sans être utilisé pour le scoring en production (voir chapitre 5). Les deux moteurs (règles et IA) partagent le même format de sortie, ce qui permet un affichage unifié dans l'interface.

2.4 Choix technologiques

Cette section motive les choix technologiques structurants de la plateforme.

2.4.1 Langage et framework back-end

Le Tableau 2.3 compare les principaux candidats pour le back-end.

Technologie	Avantages	Limites
Spring Boot 3.3 [9] (Java 17)	Support des records, du pattern matching et du text block ; Spring Security 6 avec configuration par lambda ; starter JWT via jjwt 0.12 ; starter OpenAPI (springdoc 2.6) ; écosystème mature.	Empreinte mémoire supérieure à d'autres runtimes.
Node.js / Express	Développement rapide, syntaxe JavaScript unifiée avec le front.	Support JPA/SQL moins idiomatique, écosystème SEPA moins fourni.
Django (Python)	Productivité élevée, ORM puissant.	Peu adapté à un déploiement d'entreprise Java côté client.

TABLE 2.3 – Comparaison des frameworks back-end

Spring Boot 3.3.4 avec **Java 17** a été retenu en raison de son écosystème très riche (Spring Security 6, Spring Data JPA, Spring Validation), de son support natif de PostgreSQL, de la disponibilité d'un starter OpenAPI/Swagger moderne et d'une bibliothèque JWT éprouvée pour l'authentification.



FIGURE 2.1 – Spring Boot 3 et Java 17 pour le back-end

2.4.2 Framework front-end

Le Tableau 2.4 compare les principaux frameworks JavaScript modernes.

Technologie	Avantages	Limites
React 18 + Vite [10, 11]	Composants fonctionnels, hooks, écosystème très riche, *HMR* ultra-rapide avec Vite, démarrage de projet léger (package.json minimal).	Nécessite d'assembler des librairies tierces pour couvrir des besoins avancés (routing, formulaires, DI).
Angular 17 [12]	Framework complet (routing, formulaires, HTTP, DI), TypeScript strict.	Courbe d'apprentissage plus raide, bundle initial plus lourd.
Vue.js	Documentation claire, syntaxe accessible.	Moins présent dans les grands SI d'entreprise.

TABLE 2.4 – Comparaison des frameworks front-end

React 18 servi par **Vite 5** a été retenu pour la vélocité de développement (HMR instantané), la simplicité de la configuration et la richesse de l'écosystème. Le routing est confié à react-router, et les appels REST sont centralisés via un client HTTP qui joint automatiquement le jeton d'authentification à chaque requête.



FIGURE 2.2 – React 18 et Vite pour le front-end

2.4.3 Module d'intelligence artificielle

Le Tableau 2.5 compare les options possibles pour le service IA.

Technologie	Avantages	Limites
Python 3.11 + FastAPI [13, 14]	scikit-learn (Isolation Forest, LOF), pandas, numpy, API asynchrone, documentation OpenAPI auto-générée.	Deuxième runtime à superviser.
Spring + Weka (Java)	Même runtime que le back-end.	Catalogue d'algorithmes plus restreint, performances moindres.
TensorFlow Serving	Déploiement de modèles profonds.	Surdimensionné pour des arbres isolés.

TABLE 2.5 – Comparaison des stacks IA

Python 3.11 avec **FastAPI**, **scikit-learn**, **pandas** et **numpy** a été retenu. Le service expose des endpoints pour le scoring d'un lot de paiements, l'état du modèle et son ré-entraînement. Le modèle entraîné est sérialisé et versionné. L'ensemble embarque également un pipeline *Local Outlier Factor* et un calcul de *z-score* sur le montant, combinés dans un score moyenné.

2.4.4 Base de données

Le Tableau 2.6 compare les SGBD envisagés.

SGBD	Avantages	Limites
PostgreSQL 16 [15]	Open-source, support JSONB, agrégations SQL puissantes pour les KPI, excellent support JPA/Hibernate, conteneur officiel.	Nécessite un service dédié.
Oracle Database	Robustesse éprouvée, cible historique de HR Access.	Licence propriétaire, image conteneur lourde.
H2	Base embarquée, idéale pour le développement local.	Non adaptée à la production.

TABLE 2.6 – Comparaison des SGBD

Un modèle à deux profils Spring a été mis en place : **PostgreSQL 16** pour la production (image Docker officielle) et **H2** en mémoire pour certains tests unitaires, sans altérer le code métier grâce à l'abstraction JPA/Hibernate.

2.4.5 Outils annexes et DevOps

— **Maven** pour la gestion des dépendances et du build back-end (Spring Boot, Spring Security, JPA, PostgreSQL/H2, JWT, OpenAPI, Lombok).

— **Vite + npm** pour le build front-end et le serveur de développement.

— **pip + venv** pour l'installation des dépendances Python du module IA.

— **Docker** et **docker-compose** pour l'orchestration locale et la mise en production des quatre services (base de données, back-end, module IA, front-end).

— **Lombok** pour la réduction du code répétitif côté back-end.

— **OpenAPI / Swagger UI** pour la documentation interactive de l'API.

— **Git** pour la gestion de version.

— **pytest** pour les tests du module IA et **JUnit 5 + Spring Boot Test** pour les tests back-end.

2.5 Backlog produit global

Le Tableau 2.7 regroupe les besoins fonctionnels décrits dans ce rapport, par fonctionnalité et par sprint. Les acteurs correspondent aux rôles présentés dans les chapitres suivants : administrateur, gestionnaire RH, validateur et auditeur. En l'absence du code source de l'application dans le dépôt du rapport, ce backlog constitue une synthèse documentaire ; les droits effectifs et l'état d'implémentation restent à vérifier, tandis que les priorités et les estimations proposées restent à valider.

TABLE 2.7 – Backlog produit global

ID	Fonctionnalité	User story	Priorité	Est. (pts)	Sprint
US-01	Authentification et sécurité	En tant que gestionnaire RH, je veux m'authentifier par identifiant et mot de passe afin d'accéder à mon espace de travail sécurisé.	Must	5	1
US-02	Authentification et sécurité	En tant que validateur, je veux accéder aux fonctions autorisées par mon rôle afin de respecter la séparation des responsabilités.	Must	5	1
US-03	Utilisateurs et rôles	En tant qu'administrateur, je veux créer des comptes utilisateurs afin de donner accès à la plateforme aux personnes habilitées.	Must	3	1
US-04	Utilisateurs et rôles	En tant qu'administrateur, je veux affecter un rôle à chaque utilisateur afin de définir ses responsabilités dans le processus de paie.	Must	3	1
US-05	Utilisateurs et rôles	En tant qu'administrateur, je veux désactiver un compte afin de retirer l'accès à un utilisateur qui n'est plus habilité.	Must	2	1
US-06	Journal d'audit	En tant qu'auditeur, je veux consulter les actions sensibles avec leur auteur, leur date, leur adresse IP et leur résultat afin de vérifier la traçabilité des opérations.	Must	5	1
US-07	Journal d'audit	En tant qu'auditeur, je veux retrouver les changements de statut des anomalies dans le journal afin de suivre les décisions prises pendant leur traitement.	Should	3	4
US-08	Schémas XSD	En tant qu'administrateur, je veux importer un schéma XSD pain.001 afin de prendre en charge le format de virement attendu.	Must	5	2
US-09	Schémas XSD	En tant qu'administrateur, je veux conserver plusieurs versions des schémas XSD afin de répondre aux exigences des banques destinataires.	Must	3	2
US-10	Schémas XSD	En tant que gestionnaire RH, je veux consulter les éléments et les contraintes d'un XSD afin de comprendre les données requises pour générer un fichier conforme.	Must	8	2
US-11	Transactions de paie	En tant que gestionnaire RH, je veux consulter les transactions de paie et leur statut afin de préparer les paiements à traiter.	Must	3	2
US-12	Transactions de paie	En tant que gestionnaire RH, je veux filtrer les paiements par société, période et matricules afin de constituer le lot de virements souhaité.	Must	5	2

Suite du backlog à la page suivante

ID	Fonctionnalité	User story	Priorité	Est. (pts)	Sprint
US-13	Transactions de paie	En tant que validateur, je veux approuver les transactions de paie afin que seuls les paiements autorisés soient utilisés pour la génération SEPA.	Must	5	2
US-14	Contrôles métier	En tant que gestionnaire RH, je veux détecter les IBAN invalides par le contrôle MOD-97 afin de limiter les rejets bancaires.	Must	3	3
US-15	Contrôles métier	En tant que gestionnaire RH, je veux contrôler le format des BIC selon ISO 9362 afin de vérifier les identifiants des banques bénéficiaires.	Must	2	3
US-16	Contrôles métier	En tant que gestionnaire RH, je veux détecter les montants nuls ou négatifs afin d'écarter les paiements non valides.	Must	2	3
US-17	Contrôles métier	En tant que gestionnaire RH, je veux détecter les paiements en doublon sur une période afin d'éviter de payer plusieurs fois le même salarié.	Must	5	3
US-18	Contrôles métier	En tant que gestionnaire RH, je veux détecter un montant net supérieur au brut afin de repérer les incohérences de calcul de paie.	Must	3	3
US-19	Détection par IA	En tant que gestionnaire RH, je veux détecter les montants suspects avec Isolation Forest afin de repérer les paiements atypiques non couverts par les règles métier.	Must	13	3
US-20	Détection par IA	En tant que gestionnaire RH, je veux consulter le score, le niveau de risque et les raisons d'un signalement afin d'évaluer la gravité d'un paiement suspect.	Must	5	3
US-21	Détection par IA	En tant qu'administrateur, je veux consulter l'état du modèle et déclencher son réentraînement afin de maintenir le service de détection opérationnel.	Should	5	3
US-22	Recommandations et anomalies	En tant que gestionnaire RH, je veux consulter des recommandations contextualisées pour chaque anomalie afin de choisir une action corrective adaptée.	Must	5	3
US-23	Recommandations et anomalies	En tant que gestionnaire RH, je veux consulter les anomalies détectées par les règles métier et par l'IA dans une interface commune afin de centraliser leur analyse.	Must	5	3
US-24	Recommandations et anomalies	En tant que gestionnaire RH, je veux marquer une anomalie comme corrigée ou ignorée afin de suivre son cycle de vie.	Should	5	4

Suite du backlog à la page suivante

ID	Fonctionnalité	User story	Priorité	Est. (pts)	Sprint
US-25	Virements de masse SEPA	En tant que gestionnaire RH, je veux visualiser un aperçu du XML généré afin de vérifier les informations du lot avant son enregistrement.	Must	5	2
US-26	Virements de masse SEPA	En tant que gestionnaire RH, je veux valider le XML contre le XSD sélectionné et consulter les erreurs afin de corriger les écarts de conformité.	Must	8	2
US-27	Virements de masse SEPA	En tant que gestionnaire RH, je veux générer et enregistrer un fichier pain.001 conforme à partir des transactions approuvées afin de préparer le versement des salaires.	Must	13	2
US-28	Virements de masse SEPA	En tant que gestionnaire RH, je veux télécharger le fichier XML généré afin de disposer du document à transmettre à la banque.	Must	3	2
US-29	Virements de masse SEPA	En tant que gestionnaire RH, je veux consulter l'historique des fichiers et les retélécharger à l'identique afin de retrouver les lots déjà produits.	Should	5	2
US-30	Tableau de bord KPI	En tant qu'administrateur, je veux consulter le nombre de fiches générées, les erreurs bloquantes, le taux d'erreurs et le montant total viré afin de suivre l'activité mensuelle.	Must	8	4
US-31	Tableau de bord KPI	En tant qu'administrateur, je veux visualiser l'évolution des fiches et des erreurs sur douze mois afin d'identifier les tendances de l'activité.	Should	5	4
US-32	Tableau de bord KPI	En tant qu'administrateur, je veux consulter les clients les plus concernés et les répartitions des anomalies afin de cibler les actions d'amélioration.	Should	5	4
US-33	Authentification biométrique	En tant que gestionnaire RH, je veux utiliser une authentification faciale afin de disposer d'un mode de connexion complémentaire (**Prévu** : évolution mentionnée dans le rapport, non vérifiée dans le code).	Could	8	1

La priorisation proposée suit MoSCoW : *Must* désigne les besoins indispensables, *Should* les besoins importants et *Could* les possibilités complémentaires. Les estimations en points expriment la complexité relative selon la suite 1, 2, 3, 5, 8, 13, et non une durée ; la story US-33 est rattachée au socle du sprint 1 comme évolution prévue, sans être considérée comme livrée dans ce sprint.

2.6 Diagramme de cas d'utilisation global

La Figure 2.3 présente les principales interactions des acteurs avec la plateforme SEPA, de l'authentification à la préparation et au suivi des virements de salaires. Elle reprend les rôles et les accès décrits dans les chapitres des sprints, ainsi que le module IA comme acteur système externe au back-end. Ce modèle documentaire reste à valider dans les contrôleurs et les menus React, le code de l'application n'étant pas disponible dans le dépôt du rapport.

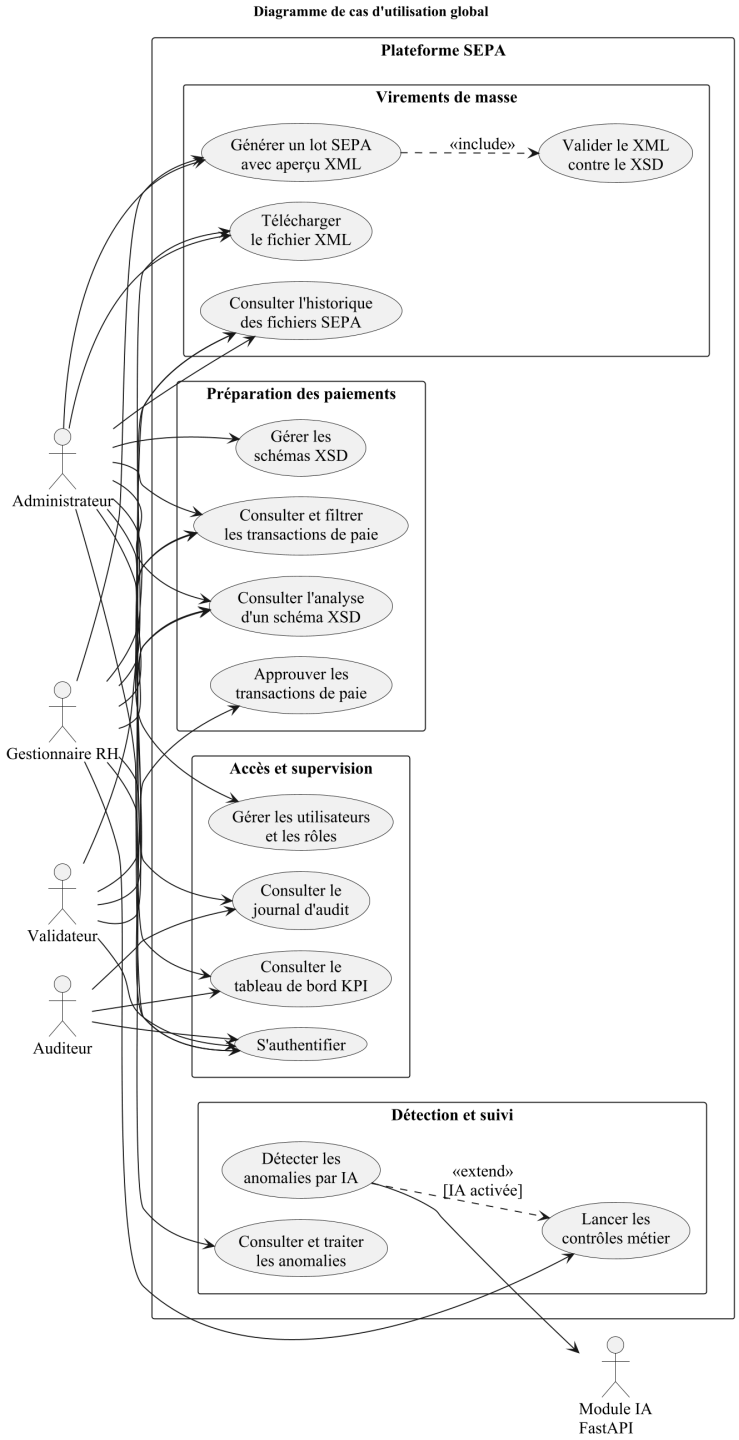


FIGURE 2.3 – Diagramme de cas d'utilisation global

Acteurs et rôles. L'administrateur gère les comptes, les rôles et les schémas XSD ; il dispose également des accès à la génération SEPA, au tableau de bord et au journal d'audit décrits dans le rapport. Le gestionnaire RH prépare les lots, examine les anomalies et produit les fichiers de virements, tandis que le validateur intervient dans l'approbation des transactions et la consultation des paiements et de l'historique. L'auditeur consulte les indicateurs et les traces d'activité ; le module IA fournit les scores et les raisons des signalements de montants suspects.

La génération inclut la validation XSD ; la détection IA complète les contrôles lorsque le service est activé. Le téléchargement suppose un fichier déjà conservé dans l'historique, sans impliquer une nouvelle génération ni une transmission automatique au système bancaire. Les autorisations exactes relatives à l'approbation et au traitement des anomalies restent à confirmer dans le code source.

2.7 Diagramme de classes global

La Figure 2.4 synthétise les principaux objets métier décrits dans les chapitres consacrés aux différents sprints. Elle rassemble les utilisateurs, les transactions, les anomalies, les traces d'audit, les schémas XSD et les fichiers SEPA. Ce modèle est construit à partir du rapport : les noms de champs, les types et les multiplicités sont descriptifs et ne constituent pas une vérification des entités JPA.

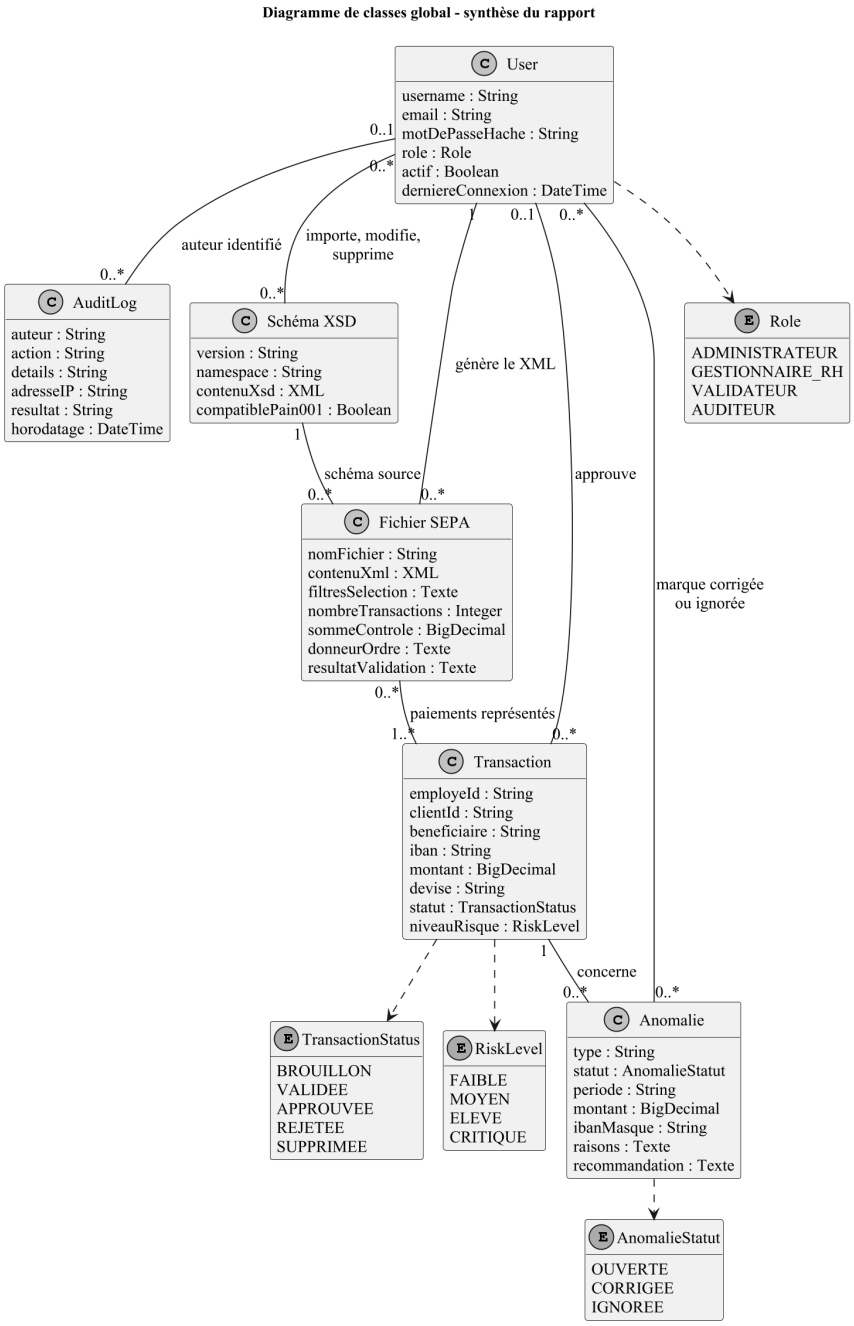


FIGURE 2.4 – Diagramme de classes global

Les transactions représentent les paiements des salariés et peuvent être associées à plusieurs anomalies, chacune accompagnée de raisons et d'une recommandation. Un fichier SEPA conserve un instantané XML des paiements ainsi que le schéma utilisé pour sa validation, tandis que le journal d'audit retrace les opérations réalisées. Les liens représentés sont conceptuels ; leur persistance sous forme de relations JPA ou d'identifiants reste à confirmer dans le code de l'application.

2.8 Diagramme de séquence global

La Figure 2.5 présente un scénario de préparation d'un lot de virements de salaires, depuis l'authentification jusqu'au téléchargement du fichier XML. Elle met en évidence les échanges entre le gestionnaire RH, l'interface React, le back-end Spring Boot, PostgreSQL et le service IA FastAPI. Cette séquence est une synthèse du parcours décrit dans le rapport, et non une trace d'exécution vérifiée dans le code.

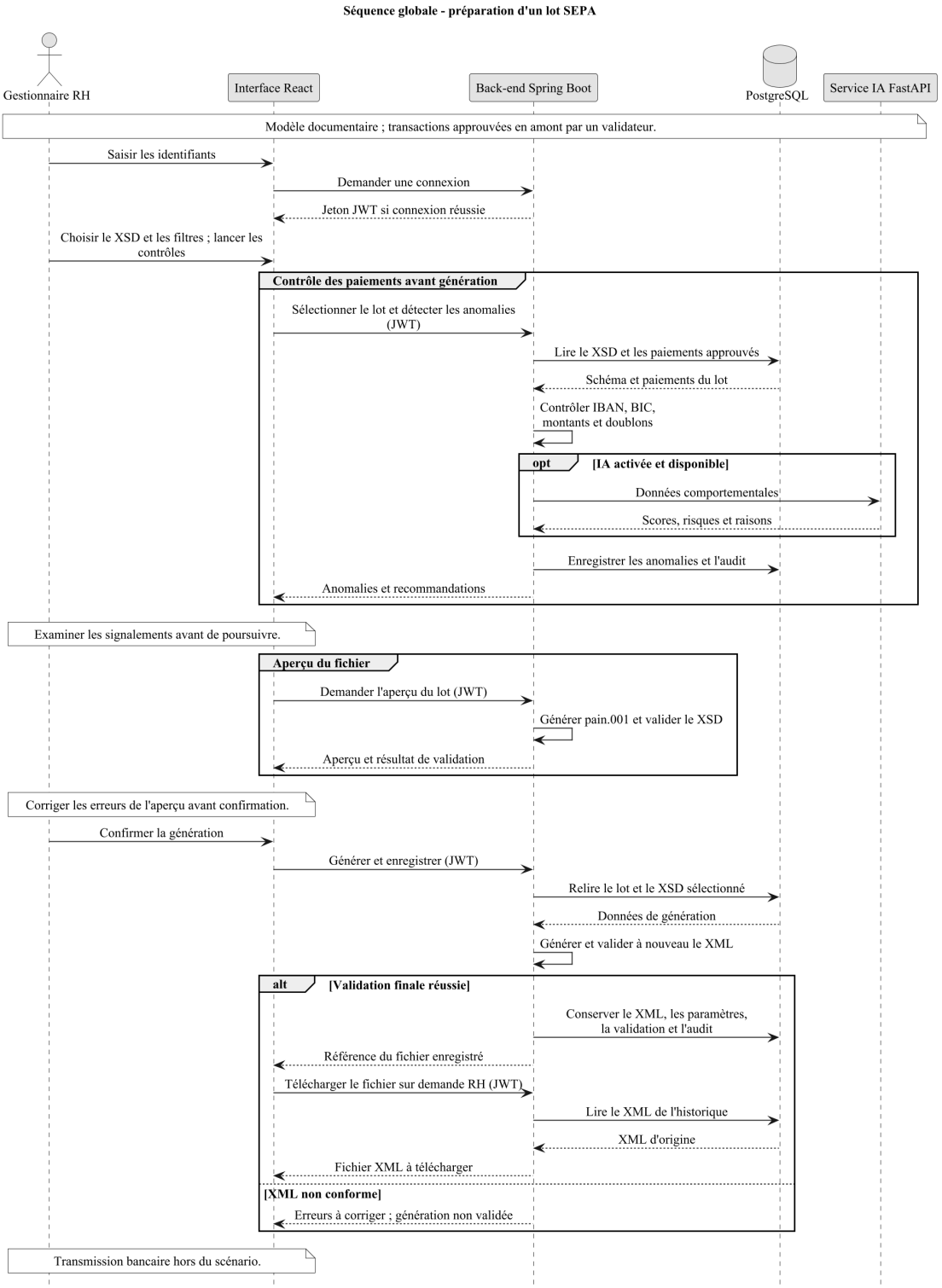


FIGURE 2.5 – Diagramme de séquence global

Les transactions sont supposées approuvées en amont par un validateur. Les contrôles métier et la détection IA fournissent des signalements à examiner, puis l'aperçu permet de vérifier le XML avant sa génération définitive. La validation XSD est renouvelée lors de l'enregistrement ; le téléchargement restitue le contenu conservé dans l'historique, sans supposer de transmission automatique à la banque.

2.9 Architecture logicielle cible

L'architecture cible retenue est une architecture web en couches, organisée autour d'un back-end Spring Boot, d'un front-end React et d'un service dédié à l'intelligence artificielle en Python, déployés sous forme de services conteneurisés. Elle est mise en œuvre selon les principes suivants (Figure 2.6) :

— **Client React** : SPA organisée par pages fonctionnelles (connexion, tableau de bord, transactions, anomalies, audit, utilisateurs, virements de masse, gestion des XSD), avec un client HTTP centralisé et un contexte d'authentification partagé.

— **Serveur Spring Boot** : contrôleurs REST exposant l'API, services métier, couche d'accès aux données, DTO explicites pour l'API, gestion centralisée des exceptions, et autorisation fine par rôle.

— **Service IA Python** : API FastAPI dédiée à la détection des montants suspects, modèle entraîné persisté et versionné.

— **Persistance** : entités JPA pour les utilisateurs, transactions, anomalies, journal d'audit, schémas XSD et fichiers générés, avec Hibernate 6.

— **Sécurité** : filtre d'authentification JWT, encodeur BCrypt, CORS restreint au front-end, routes d'authentification et de documentation publiques, politique de session *stateless*.

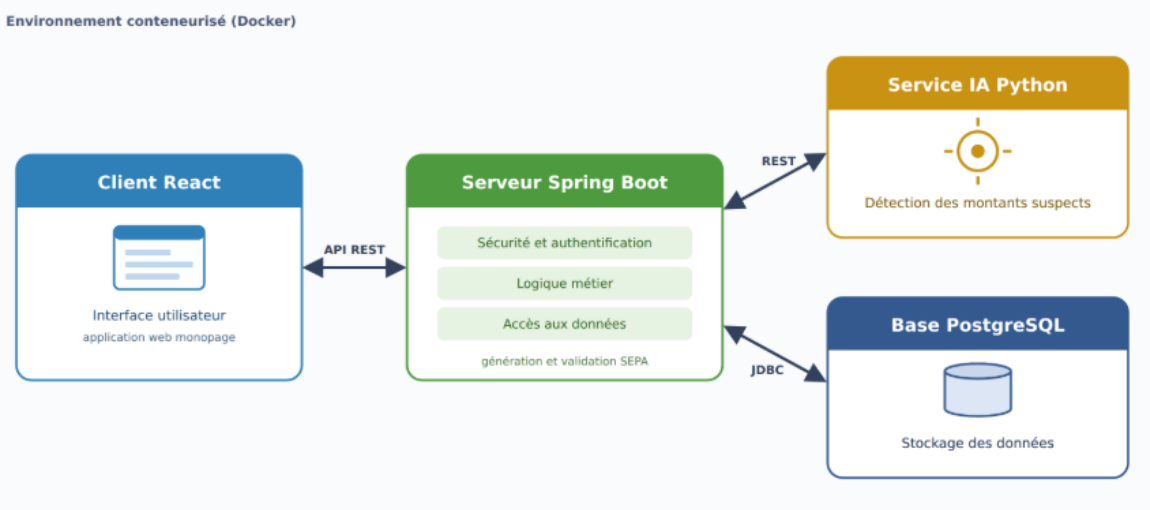


FIGURE 2.6 – Architecture logicielle en trois services conteneurisés

2.10 Conclusion

Ce sprint 0 a permis de préciser le domaine métier (paie, SEPA, ISO 20022, variantes `pain.001.001.03` et `pain.001.001.09`, IBAN/MOD-97, BIC/ISO 9362), de justifier le choix d'une détection d'anomalies *hybride* (règles symboliques Java + IA Python) et d'arrêter la pile technique :

Spring Boot 3.3 (Java 17), React 18/Vite, PostgreSQL 16, FastAPI avec scikit-learn, Maven, pip et Docker. Les chapitres suivants entrent dans le détail de la réalisation, sprint par sprint.

Chapitre 3

Sprint 1 : Authentification JWT, gestion des utilisateurs et journal d'audit

3.1 Introduction

Ce chapitre présente le premier sprint de développement, consacré au socle sécuritaire de la plateforme : l'authentification des utilisateurs par jeton JWT, la gestion des comptes et des rôles, ainsi que le journal d'audit qui trace l'ensemble des actions sensibles réalisées sur l'application. Ce sprint met en place les entités persistées, les API REST associées et les pages React correspondantes (`Login`, `Users`, `Audit`).

3.2 Backlog du sprint

Les User Stories retenues pour ce sprint sont présentées dans le Tableau 3.1.

User Story	Est. (h)	Priorité	Description
US-1 – Authentification par JWT	18	Haute	En tant qu'utilisateur, je veux m'authentifier avec un identifiant/mot de passe et recevoir un jeton JWT pour accéder aux API protégées.
US-2 – Gestion des utilisateurs & rôles	15	Haute	En tant qu'administrateur, je veux créer et désactiver des utilisateurs et leur affecter un rôle (ADMINISTRATEUR, GESTIONNAIRE_RH, VALIDATEUR, AUDITEUR).
US-3 – Journal d'audit (audit trail)	12	Haute	En tant qu'auditeur, je veux consulter la liste chronologique des actions avec l'auteur, l'horodatage, l'IP et le résultat.
US-4 – Encodage facial (option)	6	Faible	En tant qu'utilisateur, je veux pouvoir enregistrer un encodage de référence de mon visage, destiné à une future authentification biométrique.

TABLE 3.1 – Backlog du Sprint 1

3.3 Diagrammes de conception

3.3.1 Diagramme de cas d'utilisation

La Figure 3.1 présente le diagramme de cas d'utilisation associé à l'authentification et à la gestion des utilisateurs.

[Figure à produire : `fig3_1_usecase_auth`]

FIGURE 3.1 – Cas d'utilisation du sprint 1

Tous les acteurs (*Administrateur*, *Gestionnaire RH*, *Validateur*, *Auditeur*) partagent le cas d'utilisation *se connecter* qui déclenche la génération d'un jeton JWT. Seul l'*Administrateur* peut créer, consulter et désactiver des utilisateurs. L'*Auditeur* (comme l'administrateur) a accès au journal d'audit en lecture. Chaque action sensible (login, logout, génération, validation, approbation, suppression, import XSD) *inclut* un enregistrement dans le journal d'audit, modélisé par la relation «`include`».

3.3.2 Diagramme de classes

Le diagramme de classes (Figure 3.2) reprend les principales entités et services du sprint.

[Figure à produire : `fig3_2_class_auth`]

FIGURE 3.2 – Diagramme de classes du sprint 1

Le diagramme distingue les deux entités persistées (utilisateur et journal d'audit), les rôles applicatifs et les actions tracées, ainsi que les services métier (authentification, gestion des utilisateurs, audit, gestion du jeton) et les contrôleurs REST qui les exposent.

3.3.3 Diagramme de séquence – Authentification

La Figure 3.3 illustre la séquence d'appels lors de la connexion.

[Figure à produire : `fig3_3_seq_auth`]

FIGURE 3.3 – Séquence d'authentification par JWT

L'utilisateur saisit son identifiant et son mot de passe sur la page de connexion. Le mot de passe fourni est comparé à l'empreinte BCrypt stockée. En cas de succès, un jeton JWT signé est généré, la date de dernière connexion est mise à jour et une entrée de succès est ajoutée au journal d'audit ; en cas d'échec, une entrée d'échec est journalisée et une erreur 401 est renvoyée. Le jeton reçu est conservé côté client et joint automatiquement à toutes les requêtes suivantes.

3.3.4 Diagramme d'activités

La Figure 3.4 synthétise le flux d'activités de l'authentification.

[Figure à produire : `fig3_4_activity_auth`]

FIGURE 3.4 – Activités du processus d'authentification

3.4 Modèle de données

Le Tableau 3.2 liste les entités JPA du sprint et leurs principaux champs.

Entité	Champs	Contraintes
User	Identité, identifiants de connexion, rôle, statut du compte et date de dernière connexion	username et email uniques ; mot de passe hashé BCrypt
AuditLog	Auteur, action réalisée, détails, adresse IP, résultat et horodatage	action obligatoire, horodatage automatique
Role *(enum)*	ADMINISTRATEUR, GESTIONNAIRE RH, VALIDATEUR, AUDITEUR	–
AuditAction *(enum)*	Une dizaine d'actions tracées (connexion, import XSD, génération/validation/suppression de transaction, détection d'anomalie, ...)	–

TABLE 3.2 – Entités persistées du sprint 1

Au premier démarrage, un compte de démonstration est créé pour chaque rôle, avec un mot de passe par défaut uniquement destiné aux environnements de démonstration et à changer en production.

3.5 Implémentation

3.5.1 Configuration Spring Security

La classe `SecurityConfig` compose la chaîne de filtres selon les règles suivantes (adaptées à Spring Security 6) :

— CSRF désactivé car l'API est consommée uniquement en `Bearer` JWT.

— CORS restreint à `http://localhost:5173` (origine du front-end Vite).

— Les routes d'authentification et la documentation API sont publiques ; toutes les autres routes exigent un jeton valide.

— La politique de session est *sans état* : aucun cookie, le jeton JWT est transmis à chaque requête.

— Un filtre dédié intercepte chaque requête pour valider le jeton et établir le contexte de sécurité avant que la requête n'atteigne le contrôleur métier.

— L'autorisation fine est appliquée par rôle directement sur les contrôleurs sensibles (par exemple la gestion des utilisateurs réservée à l'administrateur).

Les mots de passe sont encodés avec BCrypt, qui applique un sel aléatoire et un facteur de coût configurable, résistant aux attaques par force brute.

3.5.2 Service JWT

Le jeton est signé (HMAC-SHA) avec une clé secrète externalisée et possède une durée de vie configurable (24 heures par défaut). Le service dédié sait générer un jeton pour un utilisateur

authentifié, en extraire l'identifiant, et vérifier sa validité (signature et expiration) à chaque requête.

3.5.3 Filtre d'authentification

À chaque requête, le filtre lit l'en-tête d'autorisation : si un jeton Bearer est présent, il est extrait, l'utilisateur correspondant est rechargé et, si le jeton est valide, le contexte de sécurité de la requête est établi. Les requêtes dépourvues de jeton restent anonymes et sont rejetées par les règles d'autorisation.

3.5.4 Journal d'audit

Chaque action sensible (connexion, import, génération, validation, suppression, …) est journalisée avec l'auteur (ou l'identifiant saisi en cas d'échec de connexion), l'adresse IP de la requête, le résultat (succès ou échec) et l'horodatage. La consultation du journal est réservée à l'administrateur et à l'auditeur, avec pagination côté serveur.

3.5.5 API REST du sprint

Le Tableau 3.3 récapitule les endpoints exposés par ce sprint.

Verbe	URL	Rôle requis	Description
POST	/api/auth/login	public	Authentification, retourne un jeton JWT
POST	/api/auth/face-login	public	Point d'entrée biométrique (501 par défaut)
GET	/api/users	ADMIN	Liste des utilisateurs
POST	/api/users	ADMIN	Création d'un utilisateur avec rôle
DELETE	/api/users/{id}	ADMIN	Désactivation d'un utilisateur
GET	/api/audit	ADMIN / AUDIT	Consultation paginée du journal d'audit

TABLE 3.3 – Endpoints REST du Sprint 1

3.5.6 Mesures de sécurité mises en œuvre

La propriété à sens unique de BCrypt et la présence d'un sel aléatoire garantissent qu'une lecture de la table des utilisateurs ne permet pas de reconstituer les mots de passe. La signature HMAC-SHA du jeton garantit son intégrité ; sa durée de vie limite la fenêtre d'exploitation en cas de vol de jeton. Les règles de sécurité suivantes complètent le dispositif :

— **Journal exhaustif :** toute tentative de connexion (réussie ou non) est enregistrée avec IP et horodatage.
— **Autorisation par rôle :** les routes sensibles sont explicitement restreintes par rôle.

— **Désactivation plutôt que suppression :** la suppression d'un compte le désactive (*soft delete*) en préservant la traçabilité historique.
— **Rotation de la clé JWT :** la clé est externalisée et peut être changée sans redéploiement du code.

3.5.7 Interface React

Le front-end expose trois pages principales pour ce sprint : connexion, gestion des utilisateurs (visible uniquement pour l'administrateur) et consultation du journal d'audit. Un contexte global gère le jeton et le rôle de l'utilisateur courant, et sert à protéger les routes internes selon l'authentification et le rôle attendu ; la barre de navigation masque dynamiquement les liens non autorisés.

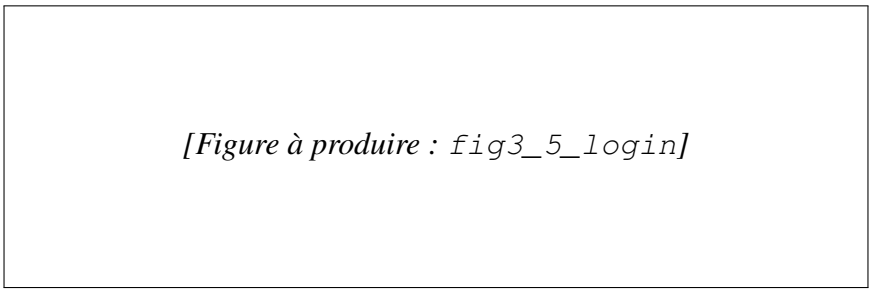


FIGURE 3.5 – Écran de connexion

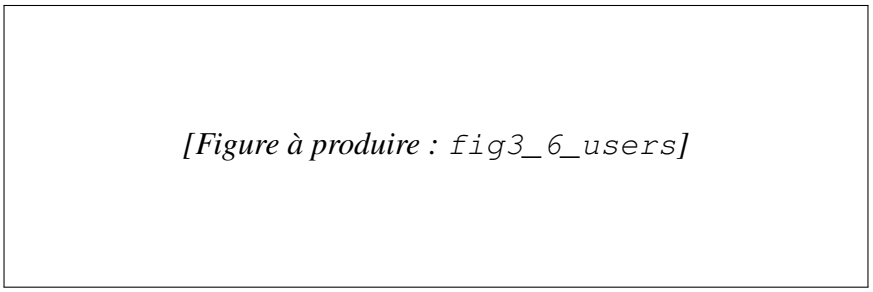


FIGURE 3.6 – Gestion des utilisateurs (vue administrateur)

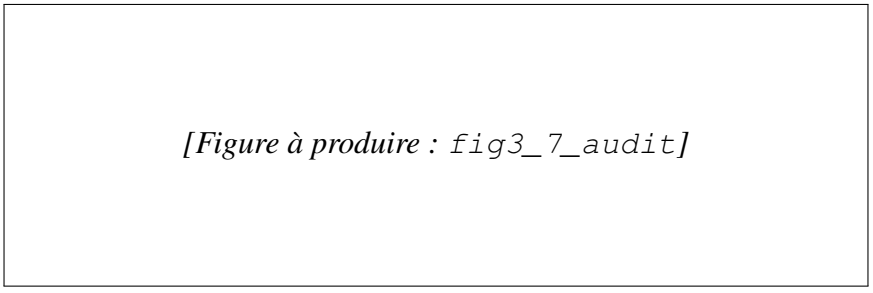


FIGURE 3.7 – Journal d'audit : traçabilité des actions

3.6 Conclusion

Ce sprint 1 a livré le socle sécuritaire de la plateforme : authentification JWT avec BCrypt, quatre rôles applicatifs portés par Spring Security 6, gestion CRUD des utilisateurs réservée à l'administrateur, et journal d'audit traçant toutes les actions sensibles avec auteur, IP, horodatage et résultat (succès/échec). Les pages React associées sont protégées par un contexte d'authentification centralisé. Le sprint suivant exploite ce socle pour implémenter la gestion dynamique des schémas XSD et la génération des fichiers SEPA.

Chapitre 4

Sprint 2 : Gestion dynamique des XSD et génération des virements de masse SEPA

4.1 Introduction

Ce chapitre présente le deuxième sprint, consacré au cœur métier du projet : la *gestion dynamique des schémas XSD* et la *génération des virements de masse SEPA*. Contrairement aux approches classiques qui ciblent une version figée du schéma, la plateforme développée permet à l'administrateur d'importer n'importe quel XSD `pain.001`, de visualiser les contraintes qu'il impose (version, namespace, obligatoires, `maxLength`, `pattern`, énumérations) et de produire ensuite un fichier conforme à ce schéma à partir des paiements sélectionnés en base. Le fichier est systématiquement validé contre son XSD source avant enregistrement dans l'historique.

4.2 Backlog du sprint

Les User Stories du sprint sont présentées dans le Tableau 4.1.

User Story	Est. (h)	Priorité	Description
US-1 – Import et versions des XSD	10	Haute	En tant qu'administrateur, je veux importer un XSD pain.001 et le versionner.
US-2 – Analyse d'un XSD	12	Haute	En tant que gestionnaire RH, je veux consulter l'analyse d'un XSD (éléments, obligatoires, contraintes de longueur, patterns, énumérations).
US-3 – Filtrage des paiements	10	Haute	En tant que gestionnaire RH, je veux filtrer les paiements par société, période et matricules.
US-4 – Aperçu + validation	15	Haute	En tant que gestionnaire RH, je veux visualiser l'aperçu coloré du XML généré et voir une mention « Conforme au XSD » ou la liste des erreurs.
US-5 – Génération et enregistrement	10	Haute	En tant que gestionnaire RH, je veux enregistrer le fichier généré dans l'historique et le télécharger.
US-6 – Historique et retéléchargement	6	Moyenne	En tant que gestionnaire RH, je veux retrouver les fichiers générés et les retélécharger à l'identique.

TABLE 4.1 – Backlog du Sprint 2

4.3 Diagrammes de conception

4.3.1 Diagramme de cas d'utilisation

La Figure 4.1 présente le diagramme de cas d'utilisation.

[Figure à produire : `fig4_1_usecase_sepa`]

FIGURE 4.1 – Cas d'utilisation du module XSD + virements de masse

L'*Administrateur* importe les XSD. Le *Gestionnaire RH* choisit un XSD compatible, visualise son analyse, filtre les paiements, lance l'aperçu, enregistre et télécharge. Le *Validateur* accède en lecture à l'historique.

4.3.2 Diagramme de classes

Le diagramme de classes (Figure 4.2) présente les classes principales du sprint.

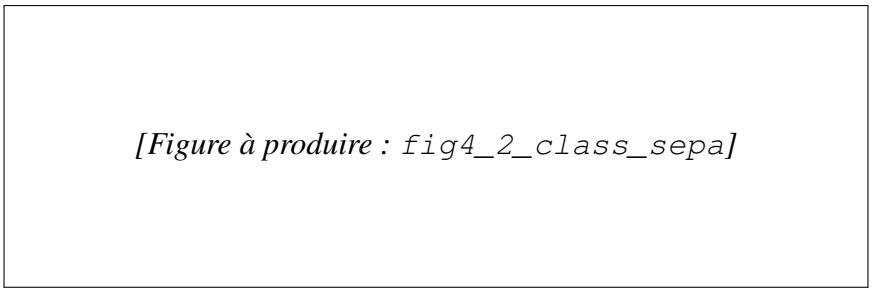


FIGURE 4.2 – Diagramme de classes du module SEPA

Les entités persistées sont le schéma XSD importé (versionné) et le fichier SEPA généré (immuable). Les composants applicatifs couvrent la lecture et l'analyse d'un XSD, la construction du XML pain.001, la validation XSD exhaustive, les contrôles métier RH (IBAN, BIC, montant, doublons) et l'orchestration du parcours complet, du filtrage des paiements jusqu'à l'historisation du fichier généré.

4.3.3 Diagramme de séquence

La Figure 4.3 illustre la génération complète d'un aperçu.

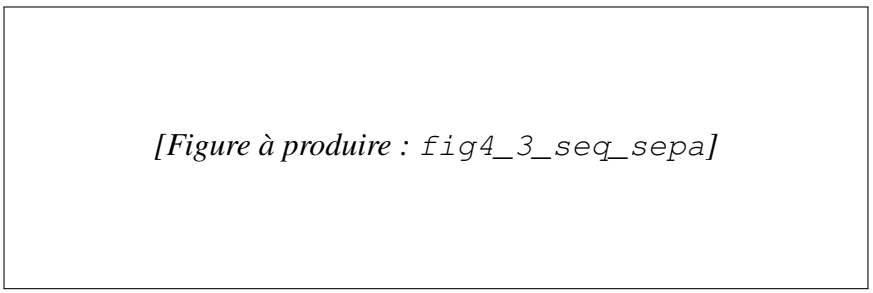


FIGURE 4.3 – Séquence de génération d'un fichier SEPA

Le client React transmet l'identifiant du XSD choisi, le donneur d'ordre et les filtres (société, période, matricules). Le service métier charge le XSD, vérifie sa compatibilité pain.001, sélectionne les transactions approuvées ou validées, applique les contrôles métier (IBAN, BIC, montant positif, doublons), puis génère le XML correspondant. Celui-ci est aussitôt validé contre le même XSD, qui remonte la liste intégrale des erreurs éventuelles (ligne, colonne, message traduit). Chaque action est tracée dans le journal d'audit.

4.4 Scénario illustratif de bout en bout

Pour fixer les idées avant de détailler chaque composant, le Tableau 4.2 reprend le parcours complet d'un gestionnaire RH souhaitant générer un fichier SEPA, en s'appuyant sur les mécanismes décrits dans les sections suivantes de ce chapitre.

Étape	Action	Description
1	Sélection d'une version XSD	Le gestionnaire RH choisit, parmi les schémas importés et marqués compatibles pain.001, la variante (001.001.03 ou 001.001.09) attendue par la banque destinataire.
2	Chargement du XSD	Le contenu du schéma sélectionné est chargé depuis la base par le service dédié.
3	Analyse de sa structure	Le schéma est analysé pour en extraire les éléments obligatoires ou répétables, leurs types et leurs contraintes (longueur, motif, énumération), restitués sous forme d'arbre dépliable côté interface.
4	Génération du XML	Les paiements filtrés (société, période, matricules) sont assemblés en un document XML pain.001 conforme à la variante choisie.
5	Validation du XML par rapport au XSD	Le document généré est immédiatement validé contre le schéma source, sans s'arrêter à la première erreur rencontrée.
6	Détection des éventuelles erreurs	Les violations (longueur, motif, élément obligatoire manquant, type incorrect) sont collectées avec leur position (ligne, colonne) et traduites en messages compréhensibles.
7	Affichage des erreurs dans l'interface	L'aperçu coloré du XML met en évidence les lignes en erreur, cliquables pour afficher le message correspondant ; une mention « Conforme au XSD » s'affiche en l'absence d'erreur.
8	Téléchargement du XML validé	Une fois la conformité confirmée, le fichier est enregistré dans l'historique puis mis à disposition au téléchargement, avec garantie de retéléchargement à l'identique.

TABLE 4.2 – Scénario illustratif de bout en bout de la gestion dynamique des XSD

4.5 Gestion dynamique des schémas XSD

4.5.1 Schéma XSD importé

Chaque XSD importé est conservé avec son nom logique, sa version (incrémentée à chaque ré-import), son contenu et des métadonnées d'import (auteur, date).

4.5.2 Analyse dynamique d'un XSD

Le composant dédié analyse un XSD pain.001 importé et produit un arbre d'éléments parcouru par le front-end, en deux temps : extraction des métadonnées (namespace, version détectée, compatibilité avec le format pain.001) puis analyse complète de la structure (éléments obligatoires ou répétables, types, et contraintes de longueur, de motif ou d'énumération portées par chaque champ). Le parsing XSD applique les précautions de sécurité standard contre les attaques par entités externes (XXE). L'arbre obtenu est affiché côté front-end dans un composant dépliable qui reprend, pour chaque nœud, son nom, son caractère obligatoire et ses contraintes.

4.6 Génération dynamique du XML pain.001

Le générateur construit le fichier XML en adaptant automatiquement sa structure à la version du schéma choisi.

4.6.1 Variantes supportées

Les deux variantes `pain.001.001.03` et `pain.001.001.09` partagent la même structure générale (en-tête du message, informations de paiement, transactions individuelles) mais diffèrent sur quelques détails de forme (représentation du code BIC et de la date d'exécution, namespace cible). Le générateur détecte la variante attendue à partir du XSD sélectionné et adapte ces éléments en conséquence.

4.6.2 Construction du fichier

Le fichier généré comporte un identifiant de message unique, un horodatage de création, le nombre de transactions et leur somme de contrôle, les informations du donneur d'ordre (nom, IBAN, BIC), puis, pour chaque paiement, un identifiant de transaction, le montant avec sa devise, les coordonnées bancaires du bénéficiaire et un libellé de virement mentionnant la période et le matricule du salarié.

4.6.3 Nettoyage SEPA

Les chaînes de caractères injectées dans le XML (noms, libellés) sont nettoyées avant insertion : les caractères accentués sont translittérés, les caractères hors du jeu autorisé par la norme SEPA sont supprimés, et chaque champ est tronqué à la longueur maximale autorisée par le XSD. Ce traitement évite les rejets bancaires dus à des caractères accentués ou spéciaux présents dans les fiches de paie.

4.7 Validation XSD avec collecte exhaustive des erreurs

Contrairement à une validation classique qui s'arrête à la première erreur, le validateur accumule *toutes* les erreurs rencontrées en conservant leur position (ligne, colonne) et en traduisant les messages du parser en expressions compréhensibles (par exemple « Longueur hors norme » plutôt que le message technique brut). Une validation syntaxique simple, sans XSD, reste disponible pour les cas où le schéma n'est pas exigé. La sécurité XXE est appliquée de la même manière que pour l'analyse du XSD.

4.8 Historique immuable des fichiers générés

Chaque génération réussie est conservée avec le nom du fichier, son contenu XML tel quel, le XSD source utilisé, les filtres ayant servi à la sélection des paiements, le nombre de transactions, la somme de contrôle, le donneur d'ordre et le résultat de la validation XSD au moment de la génération. Un re-téléchargement ultérieur renvoie ainsi exactement le fichier produit à l'origine.

4.9 API REST du sprint

Le Tableau 4.3 récapitule les endpoints exposés par ce sprint.

Verbe	URL	Rôle requis	Description
GET	/api/xsd	ADMIN	Liste des XSD importés
POST	/api/xsd	ADMIN	Import d'un nouveau XSD
GET	/api/masse/xsd	RH / VALID / ADMIN	XSD compatibles pain.001 utilisables
GET	/api/masse/xsd/{id}/analyse	RH / VALID / ADMIN	Arbre + contraintes du XSD
GET	/api/masse/filtres	RH / VALID / ADMIN	Sociétés, périodes, matricules
GET	/api/masse/paiements	RH / VALID / ADMIN	Paiements filtrés + nombre + total
POST	/api/masse/apercu	RH / ADMIN	Génère & valide sans enregistrer
POST	/api/masse/fichiers	RH / ADMIN	Génère & valide & enregistre
GET	/api/masse/fichiers	RH / VALID / ADMIN	Historique (récent d'abord)
GET	/api/masse/fichiers/{id}	RH / VALID / ADMIN	Détail (XML + erreurs + params)
GET	/api/masse/fichiers/{id}/telecharger	RH / ADMIN	Téléchargement conforme au contenu enregistré

TABLE 4.3 – Endpoints REST du Sprint 2

4.10 Interface React

La page dédiée aux virements de masse orchestre tout le parcours (Figure 4.5) : sélection du XSD (les schémas non compatibles pain.001 sont grisés), consultation de l'analyse, saisie du donneur d'ordre, filtrage des paiements (société, période, matricules), sélection des lignes avec

compteur et total, aperçu coloré du XML généré avec surlignage des erreurs, enregistrement et téléchargement. Une page dédiée à l'administration gère l'import et la liste des schémas.

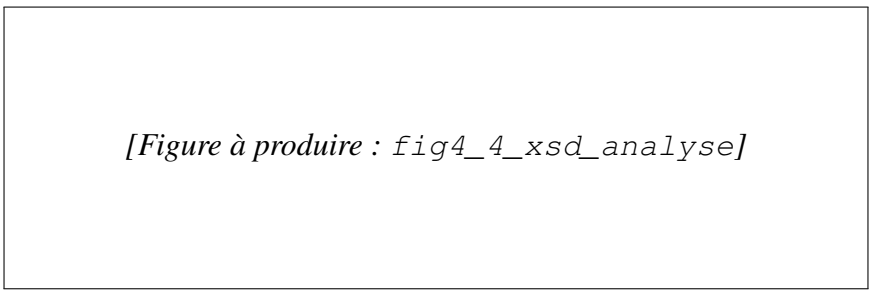


FIGURE 4.4 – Analyse d'un XSD : arbre des éléments et contraintes

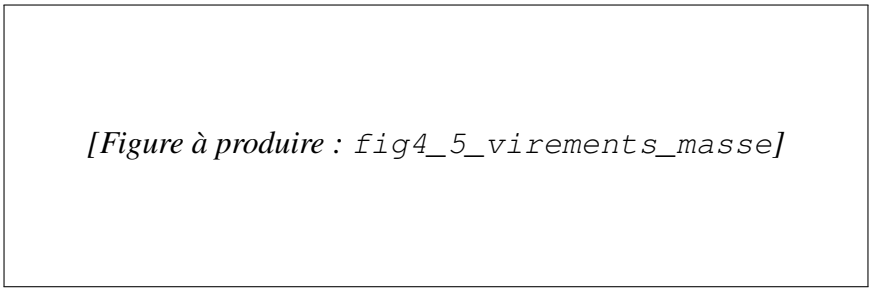


FIGURE 4.5 – Virements de masse : filtres, aperçu coloré et conformité XSD

4.11 Qualité : jeu de tests

Le sprint est couvert par des tests automatisés (JUnit 5 + Spring Boot Test) qui vérifient la lecture correcte d'un XSD pain.001 officiel, la validité du XML généré contre son schéma officiel, la collecte d'au moins deux erreurs sur un document délibérément fautif, ainsi que le filtrage, le statut par défaut des transactions sélectionnées, l'identité bit-à-bit d'un fichier re-téléchargé et l'exclusion des transactions en erreur métier.

4.12 Conclusion

Ce sprint 2 a livré le cœur métier de la plateforme : la gestion *dynamique* des schémas XSD (import, analyse, versions), la génération du XML pain.001 en deux variantes (`001.001.03` et `001.001.09`) construite via l'API DOM, la validation XSD collectant toutes les erreurs avec traduction en clair, le nettoyage SEPA systématique, la sécurité XXE et la persistance immuable des fichiers générés. L'interface React fournit un parcours bout en bout accompagné d'un aperçu coloré. Le sprint suivant complète cette chaîne par la détection d'anomalies : contrôles métier Java et modèle d'apprentissage automatique Python.

Chapitre 5

Sprint 3 : Détection hybride des anomalies (règles métier + IA Python) et recommandations

5.1 Introduction

Ce troisième sprint apporte à la plateforme sa dimension la plus différenciante : la *détection hybride* des anomalies métier. Deux moteurs coopèrent, chacun avec son terrain de prédilection : d'un côté, un **moteur de règles symboliques en Java** qui couvre les anomalies formellement caractérisées (IBAN MOD-97, BIC ISO 9362, montants, doublons, net > brut, XSD) avec explicabilité totale ; de l'autre, un **service d'intelligence artificielle Python/FastAPI** qui détecte les *montants suspects* au moyen d'un modèle *Isolation Forest* entraîné sur un vecteur de features comportementaux par paiement. Les deux moteurs partagent un format d'anomalie commun et un catalogue de recommandations lisibles, de sorte que l'interface React affiche uniformément les résultats et guide le gestionnaire RH vers une action corrective concrète.

5.2 Backlog du sprint

User Story	Est. (h)	Priorité	Description
US-1 – Contrôles métier Java	10	Haute	En tant que gestionnaire RH, je veux détecter les IBAN invalides (MOD-97), BIC mal formés, montants nuls/négatifs et doublons.
US-2 – Service IA FastAPI	18	Haute	En tant qu'administrateur, je veux un service IA qui attribue un score de risque à chaque paiement via Isolation Forest.
US-3 – Features comportementaux	10	Haute	En tant que data scientist, je veux calculer, pour chaque paiement, l'écart à la moyenne du salarié, le z-score client, la variation du mois précédent, l'IBAN changé récemment et le nombre d'IBAN partagés.
US-4 – Intégration back-end → IA	8	Haute	En tant que développeur, je veux que le back-end appelle l'IA avec un interrupteur app.ai.enabled et un timeout, sans jamais interrompre le traitement en cas d'erreur.
US-5 – Catalogue de recommandations	10	Haute	En tant que gestionnaire RH, je veux que chaque anomalie soit accompagnée d'une recommandation lisible intégrant les vraies valeurs (matricule, IBAN masqué, montant).
US-6 – Cycle de vie des anomalies	8	Moyenne	En tant que gestionnaire RH, je veux pouvoir marquer une anomalie comme CORRIGÉE ou IGNORÉE.
US-7 – Supervision du modèle	5	Moyenne	En tant qu'administrateur, je veux consulter l'état du modèle et déclencher son réentraînement.

TABLE 5.1 – Backlog du Sprint 3

5.3 Architecture de détection hybride

La Figure 5.1 présente l'architecture de détection.

[Figure à produire : `fig5_1_detection_architecture`]

FIGURE 5.1 – Architecture hybride de détection : règles Java + modèle Python

Un lot de transactions est soumis d'abord au moteur de règles métier Java. Les anomalies ainsi détectées sont persistées avec la source « règle ». Simultanément, lorsque la détection IA est activée, le lot est transmis au service d'intelligence artificielle Python qui calcule, pour chaque

paiement, un score de risque à partir de son historique et de celui du client. Les résultats sont rattachés aux transactions avec la source « IA » et le type « montant suspect ». Enfin, un moteur de recommandations produit, pour chaque anomalie (toutes sources confondues), un texte actionnable.

5.4 Moteur de règles métier Java

5.4.1 Types d'anomalies couverts

Les règles métier reconnues sont résumées dans le Tableau 5.2.

Type	Source	Signification
IBAN_INVALIDE	REGLE	IBAN manquant, mal formé ou dont la clé MOD-97 est incorrecte
BIC_INVALIDE	REGLE	BIC absent ou non conforme à ISO 9362 (8 ou 11 caractères)
MONTANT_INVALIDE	REGLE	Montant nul ou strictement négatif
DOUBLON_PAIEMENT	REGLE	Même salarié payé plusieurs fois sur la même période
IBAN_PARTAGE	REGLE	Un même IBAN apparaît chez plusieurs salariés distincts
NET_SUPERIEUR_BRUT	REGLE	Le net calculé est supérieur au brut (anomalie de calcul)
MONTANT_SUSPECT	IA	Modèle statistique : montant atypique à la lumière des features comportementaux
ERREUR_XSD	XSD	Violation de la contrainte XSD (longueur, pattern, obligatoire, type)
CTRL_SUM_INCOHEREI	XSD	NbOfTxs ou CtrlSum incohérent avec le nombre de transactions

TABLE 5.2 – Types d'anomalies supportés et sources

5.4.2 Validation IBAN MOD-97

La validation reproduit l'algorithme standard ISO 13616 : après normalisation (suppression des espaces, mise en majuscules) et vérification du format général de l'IBAN, les quatre premiers caractères sont déplacés en fin de chaîne, les lettres sont converties en chiffres, puis le nombre obtenu est réduit modulo 97 :

$$\text{IBAN valide} \iff N \bmod 97 = 1$$

5.4.3 Validation BIC ISO 9362

Le BIC est accepté s'il respecte le format normalisé ISO 9362 (8 ou 11 caractères, code banque, pays et localisation, suffixe d'agence optionnel).

5.4.4 Détection des doublons

Deux salariés payés avec le même IBAN et le même montant sur un même lot déclenchent un avertissement de doublon. Un contrôle plus riche est effectué au niveau de l'orchestration en confrontant les transactions à l'historique sur la période de paie, afin de détecter un même salarié payé plusieurs fois.

5.5 Service IA Python : détection des montants suspects

Le service de détection est un module Python indépendant, exposé par une API FastAPI dédiée.

5.5.1 Features par paiement

Chaque paiement est transformé en un vecteur de six features comportementales :

Feature	Définition
montant	Montant du paiement en euros
ecart_moyenne_salarie	$\frac{\text{montant}}{\overline{\text{historique}}_{\text{6 mois}}}$ (défaut 1 si pas d'historique)
zscore_client	z-score du montant par rapport aux autres salariés du même clientId
variation_mois_precedent	$\frac{\text{montant} - m_{t-1}}{m_{t-1}} \times 100$
iban_change_recent	1 si l'IBAN diffère du paiement précédent du salarié, 0 sinon
nb_paiements_meme_iban	Nombre de paiements du lot partageant le même IBAN

TABLE 5.3 – Features par paiement attendus par le modèle

L'ordre et les noms de ces features sont figés une fois pour toutes, garantissant la compatibilité entre le modèle persisté et toute version future du service.

5.5.2 Modèle Isolation Forest

Le modèle est une forêt d'isolement (*Isolation Forest*), une technique d'apprentissage non supervisé bien adaptée à la détection de valeurs atypiques. Ses principaux hyper-paramètres (nombre d'arbres, fraction d'anomalies attendue, seuil de score, profondeur d'historique) sont réglables par configuration.

Avant l'entraînement, les features sont normalisées. Le modèle entraîné est persisté avec sa version (horodatée) pour assurer la traçabilité des prédictions. S'il n'existe pas encore lors de la première invocation, il est entraîné sur des données synthétiques générées à cet effet.

5.5.3 Scoring et seuil de décision

Le score brut produit par le modèle est ramené à une échelle $[0, 1]$, plus lisible pour l'utilisateur métier. Un paiement est marqué *anomalie* si le modèle l'isole comme atypique ou si son score dépasse le seuil configuré. Le niveau de risque associé est dérivé par tranches (Tableau 5.4).

Niveau	Plage de score
FAIBLE	$[0; 0{,}35[$
MOYEN	$[0{,}35; 0{,}6[$
ELEVE	$[0{,}6; 0{,}85[$
CRITIQUE	$[0{,}85; 1]$

TABLE 5.4 – Niveaux de risque dérivés du score

5.5.4 Explication lisible

Lorsqu'un paiement est signalé, un libellé lisible en français est composé à partir des features à l'origine du signalement (par exemple « Montant 1,8× la moyenne des 6 derniers mois », « IBAN modifié ce mois-ci », « Hausse de 72% par rapport au mois précédent »). Cette explication prend le relais de l'opacité intrinsèque à un modèle non supervisé et rend la détection acceptable métier.

5.5.5 Endpoints du service IA

Le service expose des endpoints pour l'état du modèle, le scoring d'un lot de paiements et le réentraînement (sur des données fournies ou synthétiques). Un second pipeline expérimental, combinant plusieurs techniques de détection (Isolation Forest, Local Outlier Factor, z-score), sert de validation croisée pour comparer les approches.

5.6 Intégration back-end → IA

5.6.1 Appel du service IA

Le back-end appelle le service IA via un client HTTP configurable (URL et délai d'attente).

5.6.2 Garanties de non-régression

Le service d'intégration applique trois garanties :

— **Interrupteur global :** la détection IA peut être désactivée globalement ; dans ce cas aucun appel réseau n'est émis et le traitement métier se poursuit à l'identique.

— **Résistance aux pannes :** toute erreur (timeout, réseau, réponse vide) est capturée et journalisée sans jamais interrompre le traitement appelant.

— **Idempotence :** si une transaction précédemment signalée par l'IA ne l'est plus lors d'un nouveau passage, l'ancienne anomalie IA est purgée pour éviter les doublons.

5.6.3 Enrichissement de la transaction

En cas d'anomalie IA, la transaction est enrichie du score de risque, du niveau de risque, du type et de la source de l'anomalie, de la version du modèle utilisé, des raisons produites par le module Python et de la recommandation associée.

5.7 Entité `Anomalie` et cycle de vie

Chaque anomalie d00e9tect00e9e (quelle que soit sa source) est persist00e9e avec son type, son statut, la p00e9riode de paie, le client et le salari00e9 concern00e9s, l'IBAN masqu00e9, le montant, les raisons qui l'expliquent et la recommandation associ00e9e. Le cycle de vie est tri-états (Tableau 5.5).

Statut	Signification
OUVERTE	Anomalie nouvellement détectée, non traitée
CORRIGEE	Anomalie résolue (correction apportée à la fiche de paie)
IGNOREE	Anomalie examinée et jugée non pertinente, elle est exclue des KPI d'erreurs bloquantes

TABLE 5.5 – Cycle de vie des anomalies

L'endpoint de changement de statut accepte le nouveau statut et un commentaire optionnel. Chaque changement est tracé dans le journal d'audit.

5.8 Moteur de recommandations

Pour renforcer la valeur ajoutée métier, chaque anomalie est associée à une recommandation contextualisée qui utilise les vraies valeurs du dossier concerné (matricule, IBAN masqué, montant, période, nombre d'occurrences). Le Tableau 5.6 donne un extrait du catalogue.

Type	Recommandation (exemple)
IBAN_INVALIDE	« IBAN FR76****0189 du salarié EMP001 invalide (clé modulo 97). Demander un RIB à jour au salarié, corriger l'IBAN dans la fiche salarié, puis relancer la génération. »
BIC_INVALIDE	« BIC XYZABC12 invalide : il doit contenir 8 ou 11 caractères. Corriger le BIC dans la fiche salarié. »
DOUBLON_PAIEME	« Le salarié EMP005 est payé 2 fois sur la période 2026-08. Supprimer la fiche de paie en double (id 437) avant de relancer la génération. »
MONTANT_INVALI	« Montant de -1500,00 € pour EMP007 : un virement de salaire doit être strictement positif. Corriger le montant ou exclure la fiche du lot. »
NET_SUPERIEUR_B	« Le net (3200,00 €) est supérieur au brut (3100,00 €) pour EMP009 : vérifier le calcul des cotisations, puis corriger la fiche. »
MONTANT_SUSPEC	« Montant de 15200,00 € (Montant 1,8× la moyenne des 6 derniers mois ; IBAN modifié ce mois-ci). Vérifier primes/rappels ; si justifié, ignorer, sinon corriger. »
ERREUR_XSD	« Élément Ustrd invalide (Longueur hors norme : 140 caractères autorisés au maximum). Nettoyer le champ puis régénérer. »

TABLE 5.6 – Extrait du catalogue de recommandations

Le moteur comporte un garde-fou : toute erreur lors de la construction d'un libellé déclenche un repli sur une recommandation générique sans jamais interrompre la détection.

5.9 Configuration et supervision

Le comportement de l'intégration IA (activation, URL du service, délai d'attente, seuil de score, profondeur d'historique) est entièrement piloté par configuration externe, sans nécessiter de modification du code. Des endpoints réservés à l'administrateur permettent de superviser l'état du service IA et de déclencher un réentraînement.

5.10 Interface React

La page des anomalies affiche la liste des anomalies détectées. Chaque ligne porte un badge de niveau coloré, le type (règle/IA/XSD), le matricule, l'IBAN masqué, le montant, les raisons et la recommandation. Un panneau latéral permet de changer le statut (ouverte/corrigée/ignorée) avec un commentaire.

[Figure à produire : `fig5_2_anomalies_liste`]

FIGURE 5.2 – Interface Anomalies : règles + IA + recommandations

[Figure à produire : `fig5_3_anomalie_detail`]

FIGURE 5.3 – Détail d'une anomalie IA avec explications et recommandation

5.11 Évaluation

5.11.1 Protocole d'évaluation

Le modèle Isolation Forest de production est comparé à une règle naïve de seuil fixe, utilisée comme ligne de base simple ne tenant pas compte du comportement historique du salarié ou du client. L'évaluation s'appuie sur un jeu de données **synthétique** généré pour les besoins du développement ; aucune donnée réelle de paie n'a été utilisée pour cette évaluation (voir section 6.10 pour les limites que cela implique).

5.11.2 Métriques retenues

Trois métriques usuelles de classification binaire (paiement anormal / normal) sont retenues :

— **Précision (precision)** : parmi les paiements signalés comme anormaux par le modèle, la proportion qui le sont réellement. Une précision faible traduit un nombre élevé de faux positifs, coûteux en temps de vérification pour le gestionnaire RH.

— **Rappel (recall)** : parmi les paiements réellement anormaux, la proportion effectivement détectée par le modèle. Un rappel faible traduit des anomalies non détectées (faux négatifs), risquées du point de vue métier.

— **F1-score** : moyenne harmonique de la précision et du rappel, qui résume les deux métriques précédentes en une seule valeur lorsqu'un compromis entre faux positifs et faux négatifs doit être évalué.

5.11.3 Résultats

Le Tableau 5.7 présente le format de restitution retenu pour comparer les deux approches sur le jeu de données synthétique. Les valeurs numériques doivent être complétées à partir de l'exécution réelle du protocole d'évaluation.

Approche	**Précision**	**Rappel**	**F1-score**
Seuil fixe (ligne de base)	[À compléter avec le résultat réel]	[À compléter avec le résultat réel]	[À compléter avec le résultat réel]
Isolation Forest (production)	[À compléter avec le résultat réel]	[À compléter avec le résultat réel]	[À compléter avec le résultat réel]

TABLE 5.7 – Comparaison Isolation Forest vs seuil fixe sur le jeu de données synthétique

Le nombre de paiements et d'anomalies synthétiques utilisés pour cette évaluation **[À compléter avec la valeur réelle]**.

5.11.4 Tests automatisés

Côté Java, des tests automatisés vérifient que la désactivation de l'IA court-circuite bien le client, qu'un timeout n'empêche pas le traitement, et que la transaction est correctement enrichie en cas d'anomalie. Les contrôles métier et le moteur de recommandations sont couverts par des tests dédiés. Le nombre total de tests automatisés (back-end et module IA) est **[À compléter avec la valeur réelle]**.

5.12 Conclusion

Ce sprint 3 a doté la plateforme d'une couche de détection d'anomalies *hybride* unique : le moteur de règles symboliques Java traite avec explicabilité totale les cas formellement caractérisés (IBAN, BIC, montants, doublons, net > brut, XSD), tandis que le service IA Python dédié détecte les montants statistiquement suspects au moyen d'un Isolation Forest versionné qui exploite des features comportementales. Les deux sources partagent le même format d'anomalie, un cycle de vie trois-états et un catalogue de recommandations contextualisées. L'intégration back-end garantit, par un interrupteur global et la capture des erreurs, que le traitement métier se poursuit même en cas d'indisponibilité du service IA. Le sprint suivant exploitera ces anomalies pour construire le tableau de bord KPI et finaliser la conteneurisation.

Chapitre 6

Sprint 4 : Tableau de bord KPI, cycle de vie et conteneurisation

6.1 Introduction

Ce dernier sprint termine la plateforme en fournissant trois briques essentielles pour une mise en production effective : un **tableau de bord KPI** qui restitue, en lecture seule, les indicateurs clés de l'activité SEPA (fiches générées, erreurs bloquantes, taux d'erreurs, évolution sur douze mois, top clients, répartitions par type/source/statut) ; la finalisation du **cycle de vie des anomalies** assortie du journal d'audit étendu aux changements de statut ; et enfin une **conteneurisation** complète via `docker-compose` qui démarre les quatre services (PostgreSQL, back-end, module IA, front-end) d'une seule commande, prête pour un déploiement en recette ou en production.

6.2 Backlog du sprint

User Story	Est. (h)	Priorité	Description
US-1 – KPI mensuels	12	Haute	En tant qu'administrateur, je veux les fiches générées, les erreurs bloquantes, le taux d'erreurs et le montant total viré du mois.
US-2 – Évolution sur 12 mois	10	Haute	En tant qu'administrateur, je veux visualiser l'évolution du nombre de fiches et d'erreurs sur les douze derniers mois.
US-3 – Top clients & répartitions	10	Haute	En tant qu'administrateur, je veux le top 5 des clients à anomalies et les répartitions par type/source/statut.
US-4 – Journal d'audit du cycle de vie	6	Moyenne	En tant qu'auditeur, je veux retrouver les changements de statut d'anomalies dans le journal.
US-5 – Conteneurisation Docker	10	Haute	En tant qu'ops, je veux démarrer toute la plateforme avec docker compose up.
US-6 – Documentation API OpenAPI	4	Moyenne	En tant que développeur, je veux une page Swagger UI pour tester l'API.

TABLE 6.1 – Backlog du Sprint 4

6.3 Tableau de bord KPI

6.3.1 Définitions

Deux définitions sont centrales dans la conception du tableau de bord :

— **Fiche générée** : transaction au statut `APPROUVEE` sur la période. Ce sont les fiches réellement consommées par la génération SEPA, à l'exclusion des brouillons et des rejets.

— **Erreur bloquante** : anomalie dont le type empêche l'émission du fichier (IBAN/BIC invalide, montant invalide, doublon, net > brut, erreur XSD, `CtrlSum` incohérent) *et* dont le statut n'est pas `IGNOREE`. Les avertissements non bloquants (`MONTANT_SUSPECT` de l'IA, `IBAN_PARTAGE`) ne sont pas comptés dans ce total.

6.3.2 Indicateurs restitués

Le tableau de bord agrège les indicateurs résumés au Tableau 6.2.

Champ	Description
periode	Période affichée (mois précédent par défaut)
fichesGenerees	Nombre de transactions approuvées sur la période
clientsConcernes	Nombre distinct de clients ayant au moins une fiche générée
nbErreurs	Nombre d'erreurs bloquantes (hors anomalies ignorées)
tauxErreur	Proportion de fiches en erreur, en pourcentage
montantTotalVire	Somme des montants des fiches approuvées
anomaliesParType	Répartition des anomalies par type
anomaliesParSource	Répartition des anomalies par source (règle / IA / XSD)
anomaliesParStatut	Répartition par statut (ouvertes, corrigées, ignorées)
evolutionMensuelle	Série des 12 derniers mois (fiches, erreurs, taux)
topClientsAnomalies	Top 5 des clients les plus concernés par des anomalies

TABLE 6.2 – Indicateurs restitués par le tableau de bord

6.3.3 Calcul des indicateurs

Le service de tableau de bord est en lecture seule et repose exclusivement sur des agrégations SQL. La catégorisation des sources (règle, IA, XSD) est déduite du type de l'anomalie.
L'évolution mensuelle est reconstruite pour les 12 derniers mois (mois courant inclus) avec une interpolation à zéro pour les mois sans données, de sorte que la série renvoyée est toujours de longueur 12. Chaque indicateur est encapsulé dans un bloc `safe` qui, en cas d'erreur, enregistre un `WARN` et renvoie une valeur de repli : aucun KPI individuel ne peut mettre en défaut le tableau de bord dans son ensemble.

6.3.4 Taux d'erreur

Le taux d'erreur est le rapport, en pourcentage, entre le nombre d'erreurs bloquantes et le nombre de fiches générées sur la période (nul si aucune fiche n'a été générée), arrondi à deux décimales.

6.3.5 Endpoint REST

Verbe	URL	Rôle requis	Paramètres optionnels
GET	/api/dashboard/kpi	ADMIN / AUDIT	periode, clientId, departementId, expertId

TABLE 6.3 – Endpoint du tableau de bord KPI

Le filtre par `clientId` permet de restreindre tous les indicateurs à un seul client ; les filtres `departementId` et `expertId` sont réservés à une évolution future (acceptés par le contrôleur mais ignorés pour l'instant).

6.3.6 Interface React

Le tableau de bord compose les indicateurs sous forme de cartes et de graphiques. Les cartes du haut (fiches, clients, erreurs, taux, montant total) sont des compteurs animés. Les graphiques illustrent l'évolution mensuelle et les répartitions par type, source et statut. Un sélecteur de période et un filtre client déclenchent le rechargement des données.

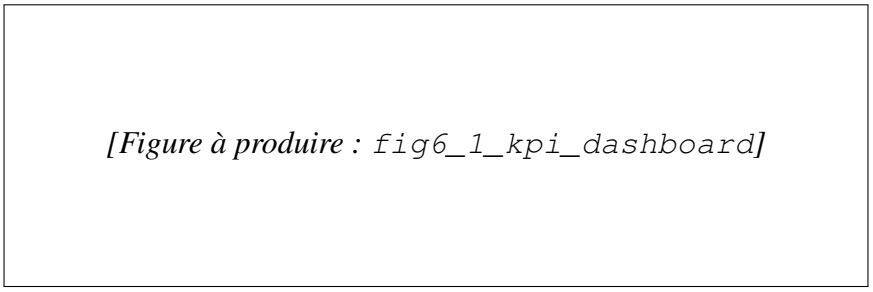


FIGURE 6.1 – Tableau de bord KPI de la plateforme

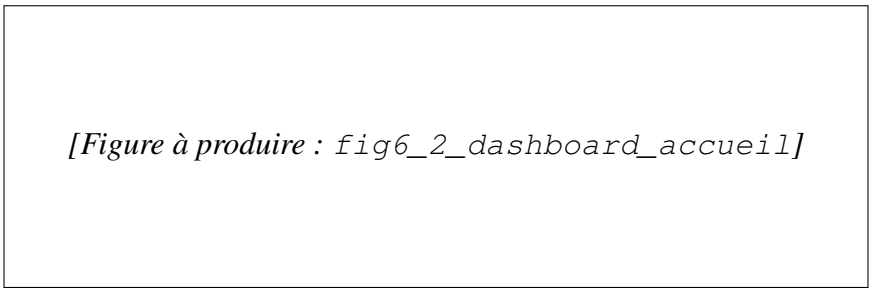


FIGURE 6.2 – Page d'accueil opérationnelle (statistiques et dernières anomalies)

6.4 Finalisation du cycle de vie des anomalies

Les changements de statut (ouverte → corrigée / ignorée) sont désormais tracés dans le journal d'audit, avec l'identifiant de l'anomalie, l'ancien et le nouveau statut, et le commentaire optionnel. Une anomalie ignorée sort automatiquement du décompte des erreurs bloquantes utilisé par les KPI, ce qui permet à l'administrateur d'améliorer la lisibilité du tableau de bord sans perdre l'information historique.

6.5 Documentation OpenAPI / Swagger

La documentation de l'API est générée automatiquement à partir des contrôleurs grâce à springdoc-openapi, et exposée via une page interactive Swagger accessible sans authentification, facilitant l'intégration par les équipes partenaires.

6.6 Conteneurisation Docker

6.6.1 Dockerfiles multi-stages

Chacun des trois services applicatifs embarque son propre Dockerfile. Le back-end utilise une construction en deux étapes qui met en cache les dépendances avant de copier le code source, réduisant les reconstructions ultérieures. Le module IA s'appuie sur une image Python légère. Le front-end sert la SPA en mode développement (ou pourrait être servi en production derrière un serveur web après compilation).

6.6.2 Orchestration `docker-compose`

Le fichier `docker-compose.yml` à la racine du projet déclare quatre services (Tableau 6.4) connectés sur le réseau par défaut, avec un volume persistant `pgdata` pour PostgreSQL et des `healthcheck` assurant un ordre de démarrage cohérent.

Service	Image	Port	Rôle
postgres	postgres :16-alpine	5432	SGBD (sepa_db, user sepa)
ai-module	build local	8000	FastAPI + Isolation Forest
backend	build local	8080	Spring Boot 3 + JWT + REST
frontend	build local	5173	React 18 + Vite

TABLE 6.4 – Services déclarés dans `docker-compose.yml`

Les relations entre services sont déclarées par la clause `depends_on` assortie des conditions `service_healthy` ou `service_started` :

- `backend` attend `postgres` (*healthy*, test `pg_isready`) *et* `ai-module` ;
- `ai-module` attend `postgres` (*healthy*) ;
- `frontend` attend `backend`.

6.6.3 Variables d'environnement

Les secrets et paramètres (connexion à la base de données, clé JWT, configuration du service IA) sont externalisés par variables d'environnement injectées par `docker-compose`. Ce mécanisme respecte le principe « douze-facteurs » et facilite la bascule entre environnements (développement, recette, production).

6.6.4 Exploitation

La plateforme se démarre, se reconstruit et s'arrête avec les commandes standard de `docker compose`. Des comptes de démonstration permettent de tester chaque rôle ; ces identifiants n'ont vocation à exister qu'en environnement de démonstration.

6.7 Points d'accès finaux

Une fois la plateforme démarrée, trois URL sont exposées à l'utilisateur (Tableau 6.5).

Service	URL
Interface web React	http ://localhost :5173
API REST Spring Boot	http ://localhost :8080/api/...
Documentation Swagger	http ://localhost :8080/swagger-ui.html
Module IA FastAPI (health)	http ://localhost :8000/health
Module IA FastAPI (docs)	http ://localhost :8000/docs

TABLE 6.5 – URL exposées par la plateforme

6.8 Synthèse qualité

Le Tableau 6.6 récapitule l'empreinte de tests livrée à la clôture du sprint.

Module	Suites de tests	Technologie
Authentification + Audit	Tests du service d'authentification et du journal d'audit	JUnit 5 + Spring Boot Test
Virements de masse	Tests de lecture du XSD, de génération (validée contre le XSD officiel) et de validation	JUnit 5
Détection d'anomalies	Tests des contrôles métier, des recommandations et de l'intégration IA	JUnit 5 + mocks
KPI dashboard	Tests du service et du contrôleur de tableau de bord	JUnit 5
Module IA Python	Tests de la détection et du calcul des features	pytest

TABLE 6.6 – Suites de tests livrées à la fin du sprint 4

6.9 Bilan quantitatif

Le Tableau 6.7 récapitule les principaux indicateurs quantitatifs de la réalisation. Les valeurs non mesurées à ce stade du rapport sont explicitement indiquées comme restant à compléter plutôt qu'estimées.

Indicateur	Valeur
Versions du schéma pain.001 prises en charge	2 (001.001.03, 001.001.09)
Rôles applicatifs	4 (Administrateur, Gestionnaire RH, Validateur, Auditeur)
Types d'anomalies supportés (règles + IA + XSD)	9 (voir Tableau 5.2)
Features comportementales utilisées par le modèle IA	6 (voir Tableau 5.3)
Services conteneurisés orchestrés	4 (base de données, back-end, module IA, front-end)
Suites de tests automatisés (familles fonctionnelles couvertes)	5 (voir Tableau 6.6)
Nombre total de tests automatisés (back-end + module IA)	[À compléter avec la valeur réelle]
Nombre de scénarios de test manuels/recette exécutés	[À compléter avec la valeur réelle]
Nombre de transactions traitées lors des essais	[À compléter avec la valeur réelle]
Nombre d'anomalies détectées lors des essais	[À compléter avec la valeur réelle]
Temps moyen de génération d'un fichier SEPA (lot type)	[À compléter avec la valeur réelle]
Temps moyen de validation XSD d'un fichier généré	[À compléter avec la valeur réelle]

TABLE 6.7 – Bilan quantitatif de la réalisation

6.10 Limites du projet

Au terme de cette réalisation, plusieurs limites peuvent être identifiées de façon académique, sans minimiser la valeur de la solution livrée :

— **Dépendance à des données synthétiques pour l'évaluation de l'IA :** le modèle Isolation Forest a été entraîné et évalué sur des données générées synthétiquement (voir chapitre 5), en l'absence de données historiques réelles et anonymisées de paie au moment de la réalisation. Les performances mesurées sur ce jeu synthétique ne garantissent pas un comportement identique sur des données de production.

— **Quantité et diversité des données disponibles :** le volume et la variété des scénarios de paiement couverts par les données synthétiques restent limités en comparaison de la diversité réelle des situations rencontrées en production (multiplicité des clients, saisonnalité, événements de paie exceptionnels).

— **Limites inhérentes aux modèles non supervisés :** l'Isolation Forest ne dispose d'aucune vérité terrain (étiquetage humain des anomalies réelles) ; il ne peut donc pas être évalué avec la même rigueur qu'un modèle supervisé, et son taux de faux positifs/négatifs en conditions réelles reste à mesurer.

— **Nécessité de données historiques réelles et anonymisées :** un déploiement en production bénéficierait d'un réentraînement sur un historique réel de paiements, anonymisé conformément aux exigences de protection des données, afin de mieux calibrer les seuils de détection.

— **Généralisation du modèle :** les features comportementales retenues (écart à la moyenne du salarié, z-score client, variation mensuelle, changement d'IBAN, paiements partageant un IBAN) ont été définies à partir de l'analyse métier du projet ; leur capacité à généraliser à d'autres contextes clients n'a pas été mesurée hors du périmètre étudié.

— **Couverture des versions ISO 20022 :** seules les variantes `pain.001.001.03` et `pain.001.001.09` du message *Customer Credit Transfer Initiation* sont prises en charge ; les autres messages ISO 20022 (prélèvements `pain.008`, relevés `camt.053`, ...) ne sont pas couverts par la plateforme actuelle.

6.11 Conclusion

Ce sprint 4 a transformé le prototype des sprints précédents en une plateforme opérationnelle : le tableau de bord KPI restitue, en lecture seule et par agrégations SQL, les indicateurs clés d'activité (fiches générées, erreurs bloquantes, taux d'erreurs, évolution sur douze mois, top clients, répartitions par type/source/statut) avec un filtre par client. Le cycle de vie des anomalies est désormais complet et chaque transition est tracée dans le journal d'audit. L'orchestration `docker-compose` démarre les quatre services (PostgreSQL, back-end, IA, front-end) en respectant les dépendances et les `healthcheck`, prête pour un déploiement en recette. Enfin, la documentation OpenAPI auto-générée simplifie l'intégration par les équipes partenaires. La plateforme est ainsi complète : de l'import d'un schéma XSD à la génération validée, de la détection hybride des anomalies à la restitution par KPI.

Conclusion Générale

Problème initial

Le versement mensuel des salaires au sein des organisations utilisatrices de la solution de paie *HR Access* repose sur la génération de fichiers de virement SEPA conformes à la norme ISO 20022. Ce processus restait, avant ce projet, exposé à plusieurs risques : des contrôles limités à la conformité structurelle du XML, l'absence de détection automatisée des anomalies métier (IBAN invalide, montants aberrants, doublons), une gestion figée d'une seule version du schéma XSD, et l'absence de traçabilité fine et de pilotage par indicateurs. Ces limites se traduisaient par des rejets bancaires, des délais de paiement et une charge de vérification manuelle importante pour les gestionnaires RH.

Solution proposée

Pour répondre à ce constat, ce projet de fin d'études, réalisé au sein de la société Sopra HR Software, a consisté à concevoir et implémenter une plateforme web qui automatise l'ensemble du cycle de traitement des flux SEPA : import et analyse dynamique des schémas XSD, génération du fichier `pain.001` ajustée à la version du schéma choisi, validation structurelle systématique, détection hybride des anomalies métier (règles expertes et intelligence artificielle), recommandations actionnables, journal d'audit complet et tableau de bord de pilotage par indicateurs clés.

Réalisation technique

Le sprint 0 a permis de préciser le domaine métier (paie, virements de salaires, norme ISO 20022, variantes `pain.001.001.03` et `pain.001.001.09`, algorithme MOD-97 de validation des IBAN, format BIC ISO 9362) et d'arrêter les choix technologiques : un back-end Spring Boot 3.3 (Java 17) sécurisé par JWT, organisé en couches et accédant à PostgreSQL via Spring Data JPA/Hibernate, un front-end React 18 servi par Vite, une base de données PostgreSQL 16, et un service d'intelligence artificielle Python/FastAPI appuyé sur scikit-learn, le tout orchestré par `docker-compose`. L'approche de détection a été retenue sous forme *hybride* : règles

symboliques Java pour l'explicabilité, modèle d'apprentissage automatique Python pour les anomalies statistiques faiblement caractérisées.
Le sprint 1 a délivré le socle sécuritaire : authentification par jeton JWT avec hachage BCrypt des mots de passe, quatre rôles applicatifs (Administrateur, Gestionnaire RH, Validateur, Auditeur) portés par Spring Security 6, gestion CRUD des utilisateurs réservée à l'administrateur, et journal d'audit traçant toutes les actions sensibles avec auteur, IP, horodatage et résultat (succès/échec). Un contexte d'authentification centralisé protège les routes internes côté front-end.
Le sprint 2 a livré le cœur métier de la plateforme : la gestion *dynamique* des schémas XSD (import, versions, analyse des contraintes), la génération du XML pain.001 en deux variantes adaptées automatiquement au schéma choisi, la validation XSD collectant toutes les erreurs avec traduction en clair, le nettoyage systématique des champs SEPA, la sécurité contre les attaques XXE, et la persistance immuable des fichiers générés garantissant un retéléchargement identique. L'interface React fournit un parcours bout en bout avec aperçu coloré et surlignage des erreurs.
Le sprint 3 a doté la plateforme d'une couche de détection d'anomalies *hybride* : un moteur de règles symboliques applique les contrôles métier (validation des IBAN et des BIC, montants, doublons, net supérieur au brut), tandis qu'un service d'intelligence artificielle dédié calcule des features comportementales par paiement et exécute, en production, un modèle Isolation Forest versionné dont le score est converti en niveau de risque. Chaque anomalie est ensuite associée à une recommandation lisible qui intègre les vraies valeurs du dossier concerné (matricule, IBAN masqué, montant, période). L'intégration back-end garantit, par un interrupteur global et la capture des erreurs, que le traitement métier se poursuit même en cas d'indisponibilité du service IA. Les anomalies suivent un cycle de vie trois-états (ouverte, corrigée, ignorée).
Le sprint 4 a transformé le prototype en plateforme opérationnelle. Le tableau de bord KPI restitue, en lecture seule et par agrégations SQL, les indicateurs clés d'activité : fiches générées, erreurs bloquantes, taux d'erreurs, montant total viré, évolution sur douze mois, top 5 des clients à anomalies, répartitions par type/source/statut. Chaque indicateur est calculé de manière défensive afin de ne jamais rompre le tableau de bord en cas d'erreur ponctuelle. La conteneurisation `docker-compose` orchestre les quatre services (base de données, back-end, module IA, front-end) avec un ordre de démarrage cohérent, prêts pour un déploiement en recette. Enfin, la documentation OpenAPI auto-générée simplifie l'intégration par les équipes partenaires.

Résultats obtenus

La solution développée répond aux principaux objectifs fonctionnels et techniques définis au début du projet : gestion dynamique des XSD, génération automatique des fichiers SEPA multi-versions, validation XSD systématique avec collecte exhaustive des erreurs, détection hybride des anomalies métier, recommandations actionnables, journal d'audit complet, tableau de bord KPI, interface web unifiée en quatre rôles et conteneurisation Docker. Elle transforme un pro-

cessus partiellement manuel et sujet aux rejets bancaires en un processus davantage industrialisé, tracé et piloté. L'évaluation quantitative détaillée du module de détection par intelligence artificielle (précision, rappel, F1-score) est présentée au chapitre 5 et reste, à ce stade, à compléter par des mesures exécutées sur le jeu de données disponible.

Limites

Les principales limites de la réalisation sont détaillées au chapitre 6 (section 6.10). Elles tiennent notamment à la dépendance du module d'intelligence artificielle à des données synthétiques pour son entraînement et son évaluation, à l'absence de vérité terrain propre aux modèles non supervisés, et à une couverture limitée aux variantes `pain.001.001.03` et `pain.001.001.09` du standard ISO 20022.

Perspectives

Plusieurs pistes d'évolution peuvent être envisagées pour prolonger ce travail :

— **Enrichissement des features IA :** intégration de données exogènes (jours fériés, événements payroll comme régularisations ou primes exceptionnelles, calendrier salarial) afin de contextualiser davantage les scores et de réduire les faux positifs.

— **Ré-entraînement supervisé :** mise en place d'une boucle de feedback : les anomalies marquées comme corrigées ou ignorées alimentent un jeu d'étiquettes qui permettrait d'évaluer, puis d'entraîner un modèle supervisé en complément de l'Isolation Forest.

— **Ouverture à d'autres messages ISO 20022 :** extension à `pain.008.001.xx` pour les prélèvements SEPA et à `camt.053` pour la réception des relevés bancaires afin d'automatiser le rapprochement des paiements effectivement exécutés.

— **Signature et archivage à valeur probante :** signature électronique du fichier XML et archivage horodaté pour répondre aux exigences légales de conservation.

— **Externalisation de l'authentification :** bascule du mécanisme JWT local vers un fournisseur d'identité OAuth 2 / OIDC et introduction du MFA (reconnaissance faciale, OTP) sur les actions sensibles.

— **Industrialisation CI/CD :** pipeline GitHub Actions ou GitLab CI, déploiement Kubernetes avec monitoring (Prometheus, Grafana) et traces distribuées (OpenTelemetry).

— **Multi-locataire (SaaS) :** isolation stricte des données par client (schéma PostgreSQL dédié ou colonne `tenant_id`) afin d'héberger plusieurs sociétés HR Access sur une même instance.

Au-delà de la liste des fonctionnalités livrées, la contribution principale de ce projet est de démontrer la faisabilité d'une architecture combinant, sur un processus critique du système

d'information RH, un moteur de règles métier explicable et un modèle d'apprentissage automatique effectivement intégré au traitement applicatif, le tout appuyé sur une gestion dynamique des schémas XSD peu répandue dans les solutions existantes examinées. La plateforme réalisée constitue une base extensible pour la génération et le contrôle des flux SEPA de paie au sein de la solution HR Access.

Références

[1] Winston W. Royce. Managing the development of large software systems, 1970.

[2] Agile Alliance. Agile software development : Principles, patterns, and practices. Agile Alliance, 2026.

[3] Scrum Alliance. Scrum guide : The definitive guide to scrum. Scrum Alliance, 2026.

[4] ISO. Iso 20022 – universal financial industry message scheme. https://www.iso20022.org/, 2026. Accessed : 2026-09-10.

[5] World Wide Web Consortium (W3C). Xml schema definition language (xsd). https://www.w3.org/standards/xml/schema, 2026. Accessed : 2026-09-10.

[6] European Payments Council. Epc sepa credit transfer rulebook. https://www.europeanpaymentscouncil.eu/, 2026. Accessed : 2026-09-10.

[7] ISO. Iso 13616 – international bank account number (iban). International Organization for Standardization, 2020.

[8] ISO. Iso 9362 – business identifier codes (bic). International Organization for Standardization, 2022.

[9] VMware, Inc. Spring boot : Convention over configuration for java-based applications. https://spring.io/projects/spring-boot, 2026. Accessed : 2026-09-10.

[10] Meta Platforms, Inc. React : A javascript library for building user interfaces. https://react.dev/, 2026. Accessed : 2026-09-10.

[11] Evan You. Vite : Next generation frontend tooling. https://vitejs.dev/, 2026. Accessed : 2026-09-10.

[12] Google LLC. Angular : The web development framework for building modern apps. https://angular.io/, 2026. Accessed : 2026-09-10.

[13] Sebastián Ramírez. Fastapi : Modern, fast (high-performance) web framework for building apis with python. https://fastapi.tiangolo.com/, 2026. Accessed : 2026-09-10.

[14] F. Pedregosa et al. scikit-learn : Machine learning in python. https://scikit-learn.org/, 2026. Accessed : 2026-09-10.

[15] PostgreSQL Global Development Group. Postgresql : The world's most advanced open source relational database. `https://www.postgresql.org/`. Accessed : 2026-09-02.

Résumé

Ce travail s'inscrit dans le cadre du projet de fin d'études réalisé au sein de Sopra HR Software en vue de l'obtention du diplôme national d'ingénieur à l'École Supérieure Privée des Sciences Appliquées et de Management.
L'objectif de ce projet consiste à concevoir et implémenter, pour la solution de paie HR Access, une plateforme de génération, de validation et de contrôle des virements de salaires au format SEPA. Contrairement aux approches limitées à une version figée du schéma, la plateforme gère dynamiquement plusieurs versions du schéma XSD `pain.001` et valide systématiquement chaque fichier généré avant son enregistrement.
La plateforme intègre une détection hybride des anomalies métier (règles expertes et intelligence artificielle), un tableau de bord d'indicateurs clés d'activité, ainsi qu'une authentification sécurisée et une gestion des rôles (administrateur, gestionnaire RH, validateur, auditeur), chaque action sensible étant tracée dans un journal d'audit.

Mots clés — HR Access, SEPA, ISO 20022, pain.001, virements de salaires, validation XSD, détection d'anomalies, intelligence artificielle, Spring Boot, React, PostgreSQL.

Abstract

This work was carried out as part of a final-year project at Sopra HR Software, with the aim of obtaining the National Engineering Degree from the Private Higher School of Applied Sciences and Management (SESAME).
The objective of this project is to design and implement, for the *HR Access* payroll solution, a platform for managing salary transfers in the SEPA format (ISO 20022, `pain.001` messages). The platform enables the dynamic import and management of multiple versions of the XSD schema, the generation of the corresponding XML file from selected payments, and its systematic validation before being stored. It also incorporates a hybrid business anomaly detection mechanism, combining an expert rule engine with an artificial intelligence service. Each detected anomaly is accompanied by an actionable recommendation. In addition, the platform provides a monitoring dashboard based on key performance indicators (KPIs), as well as fine-grained user role management.

Keywords — HR Access, SEPA, ISO 20022, pain.001, salary transfers, XSD validation, anomaly detection, artificial intelligence, Spring Boot, React, PostgreSQL.